Installation & Operations Guide

High Availability Test Tool

Release 2.14

Document Version 1.02 – Sep 14, 2018

THE BEST-RUN BUSINESSES RUN SAP

THE BEST-RUN BUSINESSES RUN SAP

SAP SE
Dietmar-Hopp-Allee 16
69190 Walldorf
Germany
T +49/62 27/74 74 74
F +49/62 27/74 74 20
www.sap.com

© Copyright 2017 SAP SE. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE. The information contained herein may be changed without prior notice.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors.

Microsoft, Windows, Outlook, and PowerPoint are registered trademarks of Microsoft Corporation.

IBM, DB2, DB2 Universal Database, OS/2, Parallel Sysplex, MVS/ESA, AIX, S/390, AS/400, OS/390, OS/400, iSeries, pSeries, xSeries, zSeries, z/OS, AFP, Intelligent Miner, WebSphere, Netfinity, Tivoli, and Informix are trademarks or registered trademarks of IBM Corporation in the United States and/or other countries.

Oracle is a registered trademark of Oracle Corporation.

UNIX, X/Open, OSF/1, and Motif are registered trademarks of the Open Group.

Citrix, ICA, Program Neighborhood, MetaFrame, WinFrame, VideoFrame, and MultiWin are trademarks or registered trademarks of Citrix Systems, Inc.

HTML, XML, XHTML and W3C are trademarks or registered trademarks of W3C®, World Wide Web Consortium, Massachusetts Institute of Technology.

Java is a registered trademark of Sun Microsystems, Inc.

JavaScript is a registered trademark of Sun Microsystems, Inc., used under license for technology invented and implemented by Netscape.

MaxDB is a trademark of MySQL AB, Sweden.

SAP, R/3, mySAP, mySAP.com, xApps, xApp, SAP NetWeaver, and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE in Germany and in several other countries all over the world. All other product and service names mentioned are the trademarks of their respective companies. Data contained in this document serves informational purposes only. National product specifications may vary.

These materials are subject to change without notice. These materials are provided by SAP SE and its affiliated companies ("SAP Group") for informational purposes only, without representation or warranty of any kind, and SAP Group shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP Group products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

Disclaimer
Some components of this product are based on Java™. Any code change in these components may cause unpredictable and severe malfunctions and is therefore expressively prohibited, as is any decompilation of these components.

Any Java™ Source Code delivered with this product is only to be used by SAP's Support Services and may not be modified or altered in any way.

Typographic Conventions

Type Style	Represents
Example Text	Words or characters that appear on the screen. These include field names, screen titles, pushbuttons as well as menu names, paths and options. Cross-references to other documentation
Example text	Emphasized words or phrases in body text, titles of graphics and tables
EXAMPLE TEXT	Names of elements in the system. These include report names, program names, transaction codes, table names, and individual key words of a programming language, when surrounded by body text, for example, SELECT and INCLUDE.
Example text	Screen output. This includes file and directory names and their paths, messages, names of variables and parameters, source code as well as names of installation, upgrade and database tools.
Example text	Exact user entry. These are words or characters that you enter in the system exactly as they appear in the documentation.
<Example text>	Variable user entry. Pointed brackets indicate that you replace these words and characters with appropriate entries.
EXAMPLE TEXT	Keys on the keyboard, for example, function keys (such as **F2**) or the **ENTER** key.

Icons

Icon	Meaning
	Caution
	Example
	Note
	Recommendation
	Syntax

Contents

CONTENTS 4

1. GETTING STARTED 6

1.1. ABOUT THIS GUIDE 6
1.2. QUICK START 6
1.3. HISTORY OF CHANGES 6

2. OVERVIEW OF THE TEST TOOL 10

2.1. TEST LANDSCAPE 10
2.2. CALL SEQUENCE 10

3. INSTALLATION PREREQUISITS 12

3.1. HARDWARE REQUIREMENTS 12
3.2. SOFTWARE REQUIREMENTS 12

4. INSTALLATION 13

4.1. COMPONENTS OF THE HA TEST TOOL PACKAGE 13
4.2. CLIENT SIDE 13
4.3. SERVER SIDE 14
4.4. UPDATE FROM PREVIOUS VERSIONS 15
4.4.1 Update from Version 2.1x 15
4.4.2 Update from Version 2.0x 15

5. EXECUTION 17

5.1. CONFIGURATION OF THE TEST RUN 17
5.1.1 Test Properties 17
5.1.1.1 File Collection Properties 17
5.1.1.2 General Test Settings 17
5.1.1.3 General Subtest Settings 19
5.1.1.4 Subtest Run Information 20
5.1.1.5 Subtest Measurement Information 22
5.1.1.6 Logon Information (Call Type Independent) 22
5.1.1.7 Logon Information (Call Type RFCStateful) 23
5.1.1.8 Logon Information (Call Type HTTPStateful) 25
5.1.1.9 Communication Traces 27
5.1.1.10 HA Events 28
5.1.1.11 Result Check 30
5.1.1.12 HTML Output 30
5.1.2 Specification of the Expected Test Result 31
5.2. STARTING THE TEST RUN 32

6. ANALYSIS OF RESULTS 34

6.1. STRUCTURE OF OUTPUT DATA 34
6.2. MAIN OUTPUT DATA OF THE TEST RUN 34
6.2.1 Summary File (Text Format) 34
6.2.2 Log File 38
6.2.3 Summary File (XML Format) 40
6.2.4 Graphical Summary File (HTML Format) 43
6.2.5 .csv File 45
6.3. OUTPUT DATA OF THE CLIENT USERS 46
6.3.1 Summary File 46
6.3.2 Response File 47
6.3.3 Error File (Overview) 47

6.3.4 Error File (Detail) 48
6.4. OUTPUT DATA OF THE HA EVENTS 48
6.4.1 Log File Written by the HA Test Tool 48
6.4.2 Output File Written by the HA Event 49

7. TEST CLASSES 50

7.1. TEST CLASS HIERARCHY 50
7.2. TESTS INCLUDED IN THE HA TEST TOOL PACKAGE 51
7.2.1 Enqueue Frame Test Class 51
7.2.2 Database Frame Test Class 51
7.2.3 Database Load Test Class 52
7.2.4 File System Frame Test Class 54
7.3. EXPECTED BEHAVIOR OF TEST CLASSES 55
7.3.1 Enqueue Frame Test Class 55
7.3.2 Database Load Test Class 55
7.3.3 File System Frame Test Class 56

8. DEINSTALLATION 57

9. SPECIAL ASPECTS 58

9.1. SECURITY CONSIDERATIONS 58
9.1.1 Client-Server Communication 58
9.1.1.1 SNC (Secure Network Communication) 58
9.1.1.2 HTTPS 58
9.1.2 Client-Side 58
9.1.3 Server-Side 59
9.2. HA EVENTS 59
9.3. SCALABILITY GUIDELINES 61
9.3.1 General Recommendations 61
9.3.1.1 Step-by-Step Approach 61
9.3.1.2 Preparing HA Events 61
9.3.1.3 Monitoring of Resource Consumption 62
9.3.1.4 Usage of Several Application Server Instances 62
9.3.2 Increasing the Load 63
9.3.2.1 Load per Call 63
9.3.2.2 Call Frequency per User 63
9.3.2.3 Number of Users 64

1. Getting Started

1.1. About this Guide

This guide gives you an overview of SAP's High Availability Test Tool and describes how to install and run it and how to analyze its results.

1.2. Quick Start

Describing the tool, its possible configurations, the analysis and the provided tests in detail is quite extensive. If you want to start using the tool quickly without taking care of all its possibilities, you can do the following:

- To get an overview on the test landscape, see chapter 2.1.
- Install the tool as described in chapters 3 and 4 ("Installation Prerequisits" and "Installation")
- For the configuration, use the file ***.../HATool/config/haQuickStartTemplate.properties***.
- Execute the tool as described in chapter 5.2 ("Execution").
- The output of the test run is written to directory ***...HATool/output/<testrunid>/<clienthost>/<calltype>/<testclass>.*** A graphical version of the summary is written to the file ***loadtest.html.*** For a textual version, have a look at the file ***loadtest.summary.***

When you have familiarized yourself with the tool, you can go into more details according to your needs by reading the corresponding chapters in the documentation.

1.3. History of Changes

The following table provides an overview of the most important changes in prior versions.

Version	Important Changes
1.00	First version of the Installation and Operations Guide for SAP's High Availability Test Tool
1.01	Chapter 5.2: Call of HA Test Tool: Separation between Windows and Unix/Linux
1.02	Add analysis of long response times - Chapter 5.1: Property noresponsethreshold - Chapter 6.2.1, 6.3.1: Section "Long response times" in summary files
1.02.00	Cleanup function for inconsistent test administration data on the server (chapter 5.1: Property specialaction=CleanAdminData) Return code of client driver indicates result of the test run analysis (chapter 6.2.2: "Log File") Improve handling of DB unavailability - Step is repeated, but no logon is done before anymore. - Internal change; not reflected in the documentation Same response code for all inconsistencies found during the test on the server. It replaces the message that probably a component was temporarily not available in the past. - Client: APP_INCONSISTENT_ERR instead of APP_PASTNOTAVAIL_ERR (chapters 6.2.1 and 6.3.2) - Server: SC_ERR_INCONSISTENCY_FOUND instead of SC_ERR_COMP_TEMP_NOT_AVAILABLE (chapter 7.3.2.2.1); same

	for SC_MSG_INCONSISTENCY_FOUND Time stamps are used in all output files (chapters 6.3.1 – 6.3.4) to ease the correlation of calls to HA incidents. The offset from the test start is not shown anymore. Document number reflects described software version level - 1.02.xx belongs to HA Test Tool v1.02. - Old document numbers were misleading (for example, 1.02 belongs to v1.01).
1.03	A database load test was introduced (chapters 7.2.2 and 7.2.3) - Test class CL_HA_DB_LOAD performs the DB load test. - CL_HA_TESTFRAME_DB is a frame test class, which is used by CL_HA_DB_LOAD. Configuration of the test run (chapter 5.1) - Chapter was restructured. - New properties o testparameters: parameters for complete test o subtestparameters: parameters for a subtest run o loopspersession: logon / logoff frequency - Meaning of property modified o specialaction: additional value ResetTestData o errorthinktimelow / errorthinktimehigh: extended to cover all disturbances on server side o tactduration: default is now 10 seconds instead of 15 Output data - Presentation of availability and responsiveness of the system in the summary file (chapter 6.2.1) - New .csv file for throughput and response time (chapters 6.1 and 6.2.5) - Examples for client output files (chapter 6.3) adapted to test run output used in chapter 6.2 Creating test classes - Extensions for INIT_TEST_EXEC (chapters 7.3.1.2 and 7.4.1.2) o Reset (without new test run or subset of subtests) o Test parameters o Scheme of method enhanced - New method INIT_SUBTEST_EXEC (chapters 7.3.1.3 and 7.4.1.3) - Information was added about fetching all subtests of a test run to method VERIFY_TEST_EXEC (chapter 7.3.1.4). - Information was added about subtest parameters to methods STEP_START, STEP_EXEC, STEP_END (chapters 7.3.1.5, 7.3.1.6, and 7.4.1.5). - Preventing call repetition is supported by the test environment now (chapters 7.3.1.5 and 7.3.2.2.2).
2.01	Test landscape - HA events were added (chapter 2.1). Installation - Customizing transport with user role (chapters 4.1 and 4.3) - Fixed directory scheme (chapter 4.2)

	- Using transaction STMS_IMPORT (chapter 4.3)
	Configuration (chapter 5.1)
	- New properties for SNC, HA events, communication traces and for including other property files
	- Removed property: outputbasedir (Output is now written to a fixed subdirectory in the HA Test Tool directory tree)
	- Modified properties
	  o subtestsparameters: new value EnqueuesPerLoop:nnn
	  o clientdelay, stepnum, duration, thinktime, errorthinktimelow and –high: upper limits introduced
	New chapters
	- Output data of HA events (chapter 6.4)
	- Expected behavior of test classes (chapter 7.3)
	- Deinstallation (chapter 8)
	- Special aspects (chapter 9)
	  o Security considerations (chapter 9.1)
	  o HA events (chapter 9.2)
	Removed chapters
	- Triggering HA-relevant events (chapter 5.3)
	- Test class development (chapters 7.3 and 7.4)

2.01 Doc 1.02	Format of HA Test Tool Package (chapters 4.1 and 4.2) - For the external delivery format .zip is used instead of .SAR - Documentation is now independent of package format - Internally the package format will be changed to .zip, too
2.01 Doc 1.03	Server side installation (chapter 4.3) - Typo corrected (‘ignore *invalid* component version’ instead of ‘.. *valid* ...’ - Additional information concerning the result of the transports Starting the Test Run (chapter 5.2) - In the call of the tool ‘– ‘ was replaced by ‘-‘ to avoid copy-paste problems when running the test (‘java –cp …’ → ‘java -cp …’)
2.02	Check of expected result - Components of HA Test Tool package (chapter 4.1) - Additional directory for result check (chapter 4.2) - New property checkfile (chapter 5.1.1; this chapter was previously chapter 5.1) - Specification of test result (new chapter 5.1.2) - Showing the results of the checks in the summary file (chapter 6.2.1) Summary file in XML format - New file added to output structure (chapter 6.1) - Description of new summary file (chapter 6.2.3) Usage of properties includefile and eventdefinitionfile at the command line (chapter 5.1.1) Update from version v2.01 (chapter 4.4) Support JRE 8 (chapter 3.2)
2.11	HTTP(s) protocol support - Chapter 2.1: Test landscape enhanced

	- Chapter 5.1.1: New properties - Chapter 9.1.1.2: Subsection of chapter 9.1: Security considerations Change of enqueue test in case of unavailability problems (chapter 7.2.1) Property clientdelay: value in milliseconds possible (chapter 5.1.1) Exit code -1 (No execution) introduced (chapters 5.1.1.2 and 6.2.2) New test class CL_HA_TESTFRAME_FILE - Chapter 7.2.4 (contains test description) - Chapter 7.3.3 (contains expected behavior) Adaption of file names of transports (chapters 4.1, 4.3 and 8) Scalability guidelines introduced (chapter 9.3) New examples of the output files (chapters 6.2 – 6.4)
2.12	HTML summary of test run - Chapter 3.2: New file d3.min.js - Chapters 4.1 and 4.2: New file haTestTool.html.template - Chapter 5.1.1.12: New property graphicslibrary - Chapter 6.2.4: Graphical Summary File (HTML Format) Update - New chapter 4.4.1: Update from Version 2.11 - Chapter 4.4.2: Update from Version 2.0x (former chapter 4.4) Chapter 6.2.5: .csv file (former chapter 6.2.4)
2.13	Variable event call parameters - Chapter 5.1.1.10: Description of corresponding test properties - Chapter 9.2: New feature is mentioned Version info for graphics library d3.min.js (Chapter 3.2) Adaption of file names of transports (chapters 4.1, 4.3) Update from version 2.1x (chapters 4.4.1, 4.4.2) instead of version 2.11
2.14	Only the client part of the HA Test Tool was modified, the server is unchanged (version 2.13) Update from previous version (chapters 4.4.1, 4.4.2) adapted Additional information about event call parameters (chapter 5.1.1.10) Introduction of ‘app_repeat_err’ (chapters 5.1.2 and 6.2.3) Show the server version of the HA Test Tool (chapters 6.2.2 and 6.2.3) Two script tags for the graphics library (chapter 6.2.4)
2.14 Doc 1.02	Activation of ICF service /sap/public/bc/icf/logoff (chapter 4.3)

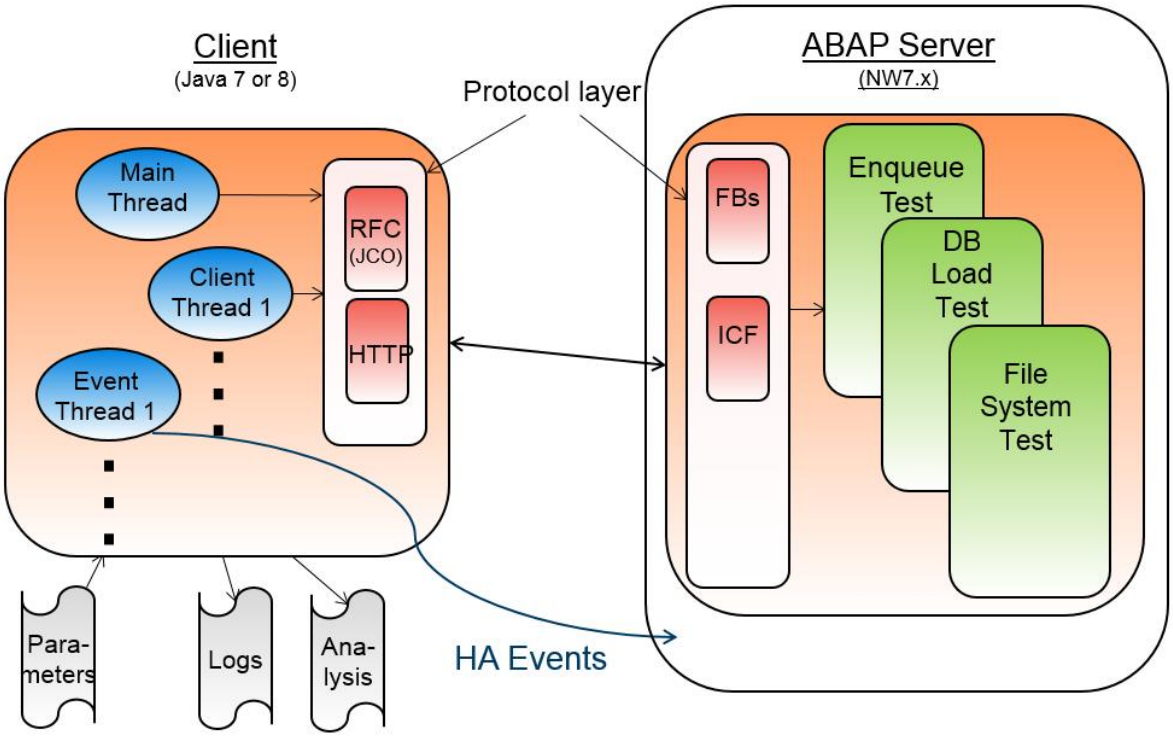
2. Overview of the Test Tool

The purpose of the SAP HA Test Tool is to check the behavior of high availability (HA) environments with SAP software under load. The tool generates load on the SAP system and allows scheduling HA-relevant events during the test run. The impact of the events on the system is documented and analyzed. This includes the availability of system components, the impact on throughput and response time and a check of the semantical correctness of the test results.

2.1. Test Landscape

The test landscape consists of a client part, which simulates a number of client users, and a server environment, which executes the calls of the client users.

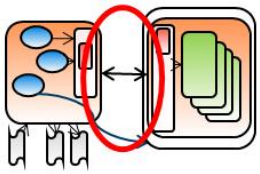
This documentation reflects the current implementation status, in which the SAP HA Test Tool is restricted to SAP Web AS ABAP 7.0 and newer versions. The client uses the RFC protocol via JCO or HTTP(S) to access server instances in order to test enqueue and database calls. HA events can be triggered by the client and effects on the server side are measured.



2.2. Call Sequence

Each test has to implement a specific interface on the server. The implementation is transparent to the client side so that the client driver is independent of the concrete test case. It just needs to know the name of the test class.

The following call sequence roughly shows the communication between client and server:



```
Main Thread: GetInfo (TestClass)
             InitTest (TestClass)
             <Run-Id> = InitSubTest (TestClass)

Client Threads: while (test not finished) {
                    Login ()
                    ExecStep (<Run-Id>, ClientNum, LoopNum, Step 1)
                     ...
                    ExecStep (<Run-Id>, ClientNum, LoopNum, Step n)
                    Logoff ()
                    LoopNum++
                }

Main Thread: EndSubTest (<Run-Id>)
             VerifyTest (TestClass)
```

➔New test classes require no code change on client side

After the client driver has done some initial calls, each client thread processes the following sequence of calls in a loop:

1. Login
2. Execute steps 1 – n
3. Logoff

If an error occurs that indicates that the server or a server component is not available, the erroneous step is repeated until it is executed successfully.

With the property ***loopspersession*** (see chapter 5.1.1), you can configure how often login / logoff calls are done.

In chapter 7 (“Test Classes”), the interface of the test classes is described in more detail.

3. Installation Prerequisits

3.1. Hardware Requirements

On server side, the resources needed by the test tool depend on the tests to be executed and on the number of parallel users.

The hardware must be sufficient for the installation of SAP Web AS ABAP 7.0 or newer and the HA environment. The setup of the HA environment is transparent for the tool.

The enqueue test, which is included, needs only little CPU, memory and database resources unless a big number of enqueues per loop is configured (see chapter 5.1.1). Additionally, you might have to adapt the size of the enqueue table (server parameter enque/table_size). With the database load test, a lot of load can be put on the server side, especially on the database. This load increases if a big number of items per order is specified. Additionally, a huge table can be generated on the database if the test runs for a long time.

In general, be careful when using a big number of users, enqueues per loop or items per order or, especially for the database load test, when executing a long test run.

On client side, the resouces needed by the test tool depend on the number of parallel users, not on the test itself. The client part is very lightweight so that a reasonable number of client users can be simulated with a normal PC.

3.2. Software Requirements

On server side, you need SAP Web AS ABAP 7.0 or newer. To conduct HA tests the system should be included in an HA environment. All operating systems and databases that are supported by SAP Web AS ABAP are allowed.

On client side, you need the following:

- JRE 7 or JRE 8
  - To get Java support, we recommend SAP JVM 7 or SAP JVM 8.
  - Can be downloaded from SAP Service Marketplace (search for SAP JVM 7.1 or SAP JVM 8.1)
- JCO 3.0
  - Can be downloaded from SAP Service Marketplace (search for SAP JCO 3.0)
- Graphics library d3.min.js (Version 3.5.x)
  - Open source library
  - The library is provided by d3js.org (see https://d3js.org/). You can use one of following links to download the library:
    - https://d3js.org/d3.v3.min.js
    - https://cdnjs.com/libraries/d3/3.5.17
  - Don't use version 4 of d3.min.js. It contains incompatible changes.
  - Is needed if a graphical view of the test run summary should be displayed
- Operating system
  - All operating systems that are supported by JCO 3.0 and SAP JVM 7 or 8 are allowed.
  - Four files are opened per simulated client user and there will be two new files per event. Especially in the case of many client users, you have to take care of the maximum allowed number of file handles configured in your operating system.

4. Installation

The components of the test package are listed and the installation of the client and server side is described. For security considerations, see chapter 9.1.

4.1. Components of the HA Test Tool Package

The HA Test Tool package contains the following files:

Client side

- Client driver (in ***HATool/bin***)
  - ***haTestTool.jar***
  - ***haTestTool.html.template***
- Templates of property files (in ***HATool/config***)
  - ***haTemplate.properties***
  - ***haQuickStartTemplate.properties***
  - ***haEventTemplate.properties***
- Template for result check and corresponding DTD file (in ***HATool/check***)
  - ***haCheckTemplate.xml***
  - ***xmlresult.dtd***

Server side

- Transports for installation (in ***HATool/server/install***)
  - Transport for programs and data
    - ***1-prog+data/K000070.D95*** and ***1-prog+data/R000070.D95***
  - Customizing transport with the role for the user
    - ***2-customizing/ K000072.D95*** and ***2-customizing/R000072.D95***
- Transport for deinstallation (in ***HATool/server/uninstall***)
  - ***K000001.BEN and R000001.BEN***

Documentation (in ***HATool/doc***)

- ***HATestTool.v214.pdf***

4.2. Client Side

Choose the location of the HA Test Tool and unpack the HA Test Tool package. It is mandatory that you do not change the directory tree used by the client driver:

.../HATool	Base directory of the HA Test Tool at client side
|	
+ ...../***bin***	Contains the driver file ***haTestTool.jar*** and the template for the HTML output (***haTestTool.html.template***)
|	
|	
+ ...../***check***	Has to contain all files to check the results of test runs (delivered with the package or written by yourself)
|	
+ ...../***config***	Has to contain all property files for the tool (delivered with the package or written by yourself)
|	

+ …../***event*** Scripts or programs for HA events
(is delivered empty; content has to be provided by yourself)

Additionally, you need to install the software for the client side described in chapter 3.2 ("Software Requirements").

For information about how to use the properties files, see chapter 5.1 ("Configuration of the Test Run").

4.3. Server Side

The server side is still on version 2.13. If this version is already installed, no installation or update is necessary. Otherwise do the following:

You have to install the transports located in ***HATool/server/install***. It is important that first the transport with programs and data (directory***1-prog+data***) and then the customizing transport (directory ***2-customizing***) is imported. For the installation, you can use the transport management system of the server (transaction ***STMS_IMPORT***).

To do so, proceed as follows (on the server, you have to be logged on as user ***<sid>adm***):

1. Import the transport with programs and data:
   a. Copy the file ***K000070.D95*** from directory ***1-prog+data*** to ***/usr/sap/trans/cofiles*** and ***R000070.D95*** to ***/usr/sap/trans/data*** on the server.
   b. Import these files with the transport management system of the server (transaction ***STMS_IMPORT***)
      i. If the transport ***D95K000070*** is not shown in the import queue, you can add it by choosing ***Extras → Other requests → Add.***
      ii. Import the request. Make sure that the the field *Target Client* contains a valid client of the target system.
2. Import the customizing transport:
   a. Copy the file ***K000072.D95*** from directory ***2-customizing*** to ***/usr/sap/trans/cofiles*** and ***R000072.D95*** to ***/usr/sap/trans/data*** on the server.
   b. Import these files with the transport management system of the server (as described above), but take care of the following:

   The content of the transport is only visible in the client specified in field *Target Client*. This has to be the client in which the test should run. Additionally, on the tab 'Options', select ***Ignore invalid component version***.

After the import the transaction ***STMS_IMPORT*** shows a screen like this:

D95K000070	SAP	Program and data transport of HA Test Tool v2.13	
D95K000072	SAP	Customizing transport for HA Test Tool v2.13	

The icon on the right side of the customizing transport means 'Does not match component version'. However, because of the ***Ignore invalid component version*** flag that you should have set, the import was not aborted.

If the imports have been successful, you see in transaction ***SE80*** the package ***SDB_HA*** that contains programs and data of the HA Test Tool. With the customizing transport, the role ***SAP_BC_HALOAD*** is created in the specified client (you can check this with transaction ***PFCG***).

As of version 2.11, the service ***hatool*** is included in the transport with programs and data to allow the access via HTTP. This service has to be activated as follows:

1. In transaction ***SICF***, enter ***hatool*** in the field *Service Name* of the filter for calling the ICF hierarchy and execute the query.
2. Right-click on the service ***hatool*** and activate the service.

For SAP NetWeaver 7.02 or above search with transaction ***SICF*** for the service name ***logoff*** and make sure that the service ***/sap/public/bc/icf/logoff*** is activated. This should normally be the case if the system was already used with HTTP.

To complete the installation on server side, you must assign the role to the user that you want to use for the tests. To do so, proceed as follows:

1. Create a user with transaction ***SU01.***
2. Assign the role ***SAP_BC_HALOAD*** to the user. This gives the user exactly as much authorizations as necessary to execute the tests.

4.4. Update from Previous Versions

4.4.1 Update from Version 2.1x

Client side:

- Use a different location for the client side.
    1. Unpack the HA Test Tool package to the new directory as described in chapter 4.2.
    2. Copy your own configuration, check and event files that you have created for version 2.1x in the corresponding directories (***…/HATool/config, …/HATool/check*** and ***…/HATool/event***) to the corresponding directories of version 2.14. You can use them without any change. Make sure that you do not overwrite files delivered with version 2.14.
    3. When starting the HA Test Tool, make sure that the file ***haTestTool.jar*** of version 2.14 is in the class path (see chapter 5.2).
- Use the same location for the client side.
    1. Back up the version 2.1x files of the subdirectories ***bin, check*** (if existing) and ***config.***
    2. Unpack the HA Test Tool package and overwrite the existing files.
    3. Changes were done in following directories:
        i. ***bin***
            1. The file ***haTestTool.jar*** was overwritten, but the old jar file is not needed anymore.
            2. As of version 2.12 there is a new file named ***haTestTool.html.template***. It is overwritten, but the old file is not needed anymore.
        ii. ***check***: If the file ***haCheckTemplate.xml*** has not been changed, no more actions are necessary. Otherwise, restore the corresponding file from the backup.
        iii. ***config***
            1. No actions concerning your own config files are necessary because version 2.14 is compatible with version 2.1x.
            2. If you have not modified the files ***haQuickStartTemplate.properties, haTemplate.properties,*** and ***haEventTemplate.properties*** from version 2.1x, no further upgrade actions are necessary.
            3. Otherwise, if you used these template files directly in your tests, you should merge your changes into the new files. However, it would be better to use your own property files for the tests to avoid this step during future upgrades.
        iv. ***doc:*** The new documentation file ***HATestTool.v214.pdf*** was added without overwriting the old one (***HATestTool.v21x.pdf***).

Server side:

No deinstallation on the server side is necessary. Just follow the instructions given in chapter 4.3.

4.4.2 Update from Version 2.0x

If you already use version 2.01 or 2.02 of the HA Test Tool, switching to version 2.14 affects the client and the server side. No mix of version 2.0x and 2.14 is allowed.

There are several possibilities to update to version 2.14:

Client side:

- Use a different location for the client side.
    1. Unpack the HA Test Tool package to the new directory as described in chapter 4.2.
    2. Copy your own configuration, check and event files that you have created for version 2.0x in the corresponding directories (***…/HATool/config, …/HATool/check*** and ***…/HATool/event***) to the corresponding directories of version 2.14. You can use them without any change. Make sure that you do not overwrite any files delivered with version 2.14.
    3. When starting the HA Test Tool, make sure that the file ***haTestTool.jar*** of version 2.14 is in the class path (see chapter 5.2).
- Use the same location for the client side.
    1. Back up the version 2.0x files of the subdirectories ***bin, check*** (if existing) and ***config.***
    2. Unpack the HA Test Tool package and overwrite the existing files.
    3. Changes were done in following directories:
        i. ***bin***: The file ***haTestTool.jar*** was overwritten, but the old jar file is not needed anymore. The file ***haTestTool.html.template*** was added.
        ii. ***check***
            1. Upgrade from version 2.01: The directory was created newly. No more upgrade actions are necessary.
            2. Upgrade from version 2.02: If the file ***haCheckTemplate.xml*** was not changed, no more actions are necessary. Otherwise, restore the corresponding file from the backup.
        iii. ***config***
            1. No actions concerning your own config files are necessary because version 2.14 is compatible with version 2.0x.
            2. If you have not modified the files ***haQuickStartTemplate.properties, haTemplate.properties,*** and ***haEventTemplate.properties*** from version 2.0x, no further upgrade actions are necessary.
            3. Otherwise, if you used these template files directly in your tests, you should merge your changes into the new files. However, it would be better to use your own property files for the tests to avoid this step during future upgrades.
        iv. ***doc:*** The new documentation file ***HATestTool.v214.pdf*** was added without overwriting the old one (***HATestTool.v20x.pdf***).

Server side:

No deinstallation on the server side is necessary. Just follow the instructions given in chapter 4.3.

5. Execution

5.1. Configuration of the Test Run

5.1.1 Test Properties

For a quick introduction, you can have a look at file ***haQuickStartTemplate.properties*** in subdirectory ***config*** of the HA Test Tool base directory. It contains a mimimum set of properties that are necessary to make test runs with the tool.

The configuration of the test run is specified by a set of <key>=<value> pairs (no spaces are allowed in the command line), which are used as input of the test driver on client side.

The following properties are supported:

5.1.1.1 File Collection Properties

Property	***file***=<property file>
Description	The file can contain all properties below so that no additional <key>=<value> pairs are necessary in the command line. The installation contains the property files. ***haTemplate.properties*** and ***haQuickStartTemplate.properties***, which can be used as a basis for the configuration. ***haTemplate.properties*** contains all available properties, whereas ***haQuickStartTemplate.properties*** is restricted to a minimum set that is sufficient to make first runs with the tool. Properties that are specified additionally in the command line take precedence over their settings in the property file. If the properties ***includefile*** and ***eventdefinitionfile*** (see below) are specified in the command line, this is valid for their content, too. It is planned that a test run can be composed of several subtests. However, in the current version of the tool, only one subtest per test run is allowed. For the ease of use, there is currently one combined property file for the test run and the subtest.
Required	No
Values	Character string The property file must be located in subdirectory ***config*** of the base directory of the HA Test Tool (***HATool***). Therefore, a directory name must not be specified.
Default	---

Property	***includefile***=<property file>
Description	If different tests with different property files should be executed, there are often groups of properties that have the same values (for example, the logon information). In order to avoid specifying these properties several times, you can group them into one property file and include it via property ***includefile***. Several include files can be used. Even nested include files are allowed.
Required	No
Values	Character string The property file must be located in subdirectory ***config*** of the base directory of the HA Test Tool (***HATool***). Therefore, a directory name must not be specified.
Default	---

5.1.1.2 General Test Settings

Property	***testrunid***=<ID of the test run>
Description	Each test run must have a unique ID. This property is optional. If it has not been set, the timestamp of the test start is used (YYYYMMDDHHMMSS).
Required	No
Values	Character string; up to 30 characters
Default	Timestamp of the test start (YYYYMMDDHHMMSS)

Property	***resettestdata***=<reset old test data>
Description	It is specified whether data of old test runs should be deleted on the server. Reset could be limited by setting the property ***testparameters=ResetExcluded*** or ***...=ResetIncluded*** (see below).
Required	No
Values	1: Data of old runs of this test class is deleted on the server. 0: Old test data is not deleted.
Default	1

Property	***testparameters***=<parameters for test run>
Description	Test parameters that are passed on to the server during the initialization phase of the test run. They belong to the test class specified below. The right-hand side must not contain white space characters. Syntax: testparameters=<param1>[:<value1>[,value2, ...][;<param2>…]
Required	No
Values	Currently supported parameters: - ***ResetExcluded*** :id1[,id2...] The list of internal subtest IDs specifies the subtests whose data should not be reset. An ID is unique for a subtest run and is shown in the output files ***loadtest.log*** and ***loadtest.summary*** (chapter 6.2). - ***ResetIncluded*** :id1[,id2...] The list of internal subtest IDs specifies the subtests whose data should be reset. In both cases the IDs must belong to subtests of the test class specified below by the property ***testclass***. These parameters are used in combination with the properties ***resettestdata*** and ***specialaction=ResetTestData***. If resettestdata=0 and special action ResetTestData has not been set, ***ResetExcluded*** or ***ResetIncluded*** will be ignored. It is not allowed to set both parameters together.
Default	---

Property	***specialaction***=<special action>
Description	A special action means that no test will be performed, but a special task should be executed.
Required	No
Values	The following actions are currently supported: - ***specialaction***=***CleanAdminData*** Clean administration data. This action is used if it is erroneously indicated that a test is running for the test class specified in the property ***testclass*** below. Such a scenario can happen if the server was restarted during a test run and was not up and running when the test finished on client side. Therefore, the administration data of the test run could

	not be adapted in the database on the server side. Other potential causes are severe crashes on the client or server side during a test run. ***CleanAdminData*** is used to set the test administration data back to a consistent state, which means that it is no longer indicated that a test is running for the test class. No test run is executed; therefore, the properties for test and measurement information are not needed. Use this action only if you are absolutely sure that no test is running for the test class. Since this action is a severe intervention, it has to be confirmed during the run. If the confirmation is not done, -1 (no execution) is returned. - ***specialaction=ResetTestData*** Reset data of previous test runs. This action is used if data of old test runs should be deleted without executing a new test. The reset of data is restricted to runs of the test class specified in the property ***testclass*** below and can be further limited by setting the property ***testparameters=ResetExcluded*** or ***...=ResetIncluded*** (see above).
Default	---

5.1.1.3 General Subtest Settings

Property	***clienthost***=<host name>
Description	Name of the machine of the test driver.
Required	No
Values	Character string; up to 30 characters
Default	If this property has not been set, the name is calculated automatically.

Property	***testclass***=<name of test class>
Description	Test class that should be executed on server side.
Required	Yes
Values	***CL_HA_TESTFRAME_ENQ***: Enqueue test class ***CL_HA_TESTFRAME_FILE***: File system test class ***CL_HA_DB_LOAD***: Database test class (***CL_HA_TESTFRAME_DB***): Frame test class for database test; should normally not be called directly. For more details about the test classes see chapter 7.
Default	---

Property	***calltype***=<protocol>
Description	Type of protocol that is used to call the server.
Required	Yes
Values	***RFCStateful***: RFC protocol via Jco ***HTTPStateful***: HTTP / HTTPS protocol (differentiation via property ***https***) “Stateful” means that all calls between logon and logoff are processed in the same session context.
Default	---

Property	***subtestparameters***=<parameters for subtest run>
Description	Special parameters for the subtest run. They are passed on to the server during the initialization phase of the subtest. The right-hand side must not contain white space characters. The syntax can be defined by each test class individually, but for the following syntax the test framework on the server provides support for parsing and storing the parameters: subtestparameters=<param1>[:<value1>[,value2, ...][;<param2>…]
Required	No
Values	The following values are currently allowed depending on the test class: - ***EnqueuesPerLoop***:nnn Can be used for ***CL_HA_TESTFRAME_ENQ*** (Enqueue test class). You can specify the number of enqueues per call sequence loop. A value between 1 and 10000 is allowed. - ***ItemsPerOrder***:nnn Can be used for ***CL_HA_DB_LOAD*** (DB load test class). You can specify the number of order items per order. A value between 1 and 10000 is allowed. Other test classes ignore this parameter.
Default	***EnqueuesPerLoop***:1 (for test class ***CL_HA_TESTFRAME_ENQ***) ***ItemsPerOrder***:10 (for test class ***CL_HA_DB_LOAD***)

5.1.1.4 Subtest Run Information

Property	***clientnum***=<number of client users>
Description	Number of simulated client users.
Required	Yes
Values	1 - 10000 (check carefully how many users your system can handle)
Default	---

Property	***clientdelay***=<delay for start of clients users>
Description	Delay in seconds between the start of two client users. As of version 2.11 the value can also be specified in milliseconds to allow faster startup, especially if many client users are simulated. Use suffix 'ms' in that case. Example: clientdelay = 100ms
Required	No
Values	0 - 10
Default	1

Property	***stepnum***=<number of steps>
Description	Number of test steps that are repeated during a loop.
Required	Yes
Values	1 - max. number of steps allowed by the test class. If there is no dedicated limit by the test class, not more than 100 steps are allowed. Test class CL_HA_TESTFRAME_ENQ: allowed range 2 - 100 (2 - 5 is sufficient) Test class CL_HA_HA_DB_LOAD: allowed range 2 – 5 Test class CL_HA_TESTFRAME_FILE: allowed range 1 - 100 (up to 5 is sufficient)
Default	---

Property	***loopspersession***=<number of loops per session>
Description	Number of loops executed per session. A session is surrounded by logon and logoff. Example: stepnum=3, loopspersession=2 Then following sequence is executed: login step1 step2 step3 step1 step2 step3 logoff This sequence is repeated during the runtime of the test until the client stops.
Required	No
Values	Number (not negative) 0: All steps of a client of the whole subtest run are processed in one session.
Default	1 (corresponds to the test behavior before the property was introduced in v1.03)

Property	***duration***=<test duration>
Description	Duration of the high load phase of the test in seconds. This period starts after all client users are up and running and have at least executed one loop of their work.
Required	Yes
Values	1 - 864000 (= 10 days)
Default	---

Property	***durationafterevents***=<test duration after last event>
Description	Duration of the high load phase in seconds after the last event finished. If this parameter has not been set, the property ***duration*** (see above) is used without taking care of the events. If it has been set, the highload phase ends as defined by the values of ***duration*** or ***durationafterevents***, whatever occurs first.
Required	No
Values	Number (not negative) The value of ***durationafterevents*** has to be lower than the value of ***duration***.
Default	---

Property	***thinktime***=<step think time>
Description	Think time before a step is executed (in seconds). Each client user waits this time before the next step is executed.
Required	No
Values	0 - 60
Default	5

Property	***errorthinktimelow***=<minimum think time during disturbance of the system>
Description	During or directly after a period of disturbance of the system, calls of the clients often return in a very small time range. To vary the point in time when the next calls are issued, you can specify a range for the think time via the properties ***errorthinktimelow*** and ***errorthinktimehigh***. The think time is calculated randomly within this range. There are several conditions under which ***errorthinktimelow*** and ***–high*** are used: - Server not available: Think time for logon calls, which are repeated until the server is up again

	- Errors because a component of the system is not available: Think time for step calls that are repeated until they are processed successfully. - Long response times: Think time for calls made directly after a call with a long response time. - Server / system component available again: Think time for calls made directly after a period of unavailability of the server or of system components
Required	No
Values	0 - 60
Default	Value of property ***thinktime*** (see above).

Property	***errorthinktimehigh***=<maximum think time during disturbance of the system>
Description	See description of ***errorthinktimelow*** above. If ***errorthinktimehigh*** is higher than ***errorthinktimelow***, a random value is calculated in this range. This can avoid system overload after a disturbance during the test run.
Required	No
Values	0 - 60 The value must be at least as big as ***errorthinktimelow***.
Default	Value of property ***errorthinktimelow*** (see above).

5.1.1.5 Subtest Measurement Information

Property	***tactduration***=<duration of a tact>
Description	Duration of a tact in seconds. The high load phase is split into tacts. For each tact, the throughput and the average response time is measured. This indicates how fast a system returns back to its normal behavior after a restart.
Required	No
Values	Positive number that has to be lower or equal to ***duration***. Recommendation: The value should be a divisor of the duration.
Default	10

Property	***noresponsethreshold***=<threshold for long response times>
Description	Check whether the server does not respond for a certain amount of time although it is up and running. For the analysis, calls to the server are used that last at least as long as specified in the threshold property (in seconds).
Required	No
Values	Non negative number that must not be bigger than ***duration***. 0: No check is done.
Default	10

5.1.1.6 Logon Information (Call Type Independent)

Property	***systemclient***=<logon client>
Description	For SAP Web AS ABAP: client number for logon (for example, 001).
Required	Yes
Values	3 digit number
Default	---

Property	***user***=<user name>
Description	Logon user on the server.
Required	Depends on the authentification mode of the protocol Yes: - RFC without SNC (***sncmode***=0) or with SNC (***sncmode***=1), but without single sign on (***sncsso***=0). - HTTP or HTTPS (https=1) and server authentication is done (no keystore is set) without specifying user and password on server side in transaction SICF for service 'hatool'. No: All other authentication modes
Values	Character string
Default	---

Property	***password***=<password>
Description	Password for logon user on the server. Hint: If you do not want to specify the password in the properties file, you can add it directly to the command line, like all other properties.
Required	Same as for property ***user***.
Values	Character string
Default	---

Property	***language***=<language>
Description	Language used for logon (for example: en).
Required	Yes
Values	Character string of length 2
Default	---

5.1.1.7 Logon Information (Call Type RFCStateful)

Logon can be done in two ways, via the message server or the application server. If information for both ways is provided, connect is done via the message server.

Logon via message server:

Property	***msgserverhost***=<host name>
Description	Host name of the message server.
Required	Only if RFC should connect via the message server.
Values	Character string
Default	---

Property	***msgserverport***=<port number>
Description	Port number of the message server.
Required	This information is only necessary if there is no entry for sapms<sid> in /etc/services or C:\Windows\System32\drivers\etc\services.
Values	Digits representing the port number
Default	---

Property	***logongroup***=<logon group>
Description	Logon group to be used.
Required	No
Values	Character string If no logon group was created on the server, ***SPACE*** has to be used.
Default	PUBLIC

Property	***systemid***=<SID>
Description	SID of the system.
Required	This property is only mandatory if ***msgserverport*** has not been set.
Values	Character string of length 3
Default	---

Logon via application server:

Property	***appserverhost***=<host name>
Description	Host name of the application server to which RFC should connect directly.
Required	Only if RFC should connect to the application server directly.
Values	Character string
Default	---

Property	***systemnumber***=<system number>
Description	Number of the system (for example, 00).
Required	Only if RFC should connect to the application server directly.
Values	2 digit number
Default	---

Secure Network Communication (SNC) for RFC:

These properties are only taken into account if ***calltype=RFCStateful***.

Property	***sncmode***=<should SNC be used>
Description	For security reasons, RFC can be done via SNC. Without SNC, authentication is done via user and password (see properties above); no encryption is used.
Required	No
Values	0: no SNC 1: SNC
Default	0

The following SNC properties are only used if sncmode=1:

Property	***snclibrary***=<SNC library>
Description	SNC library on client side (including the complete path). The library is not part of the distribution of the HA Test Tool. It has to provide the GSS-API V2 standard interface and has to fulfill the SAP requirements for SNC libraries.
Required	Yes, if RFC with SNC should be used

Values	Character string
Default	---

Property	***sncmyname***=<SNC name>
Description	SNC name of client (like p:CN=xxxxxxx, O=yyyyy, C=zz).
Required	No.
Values	Character string
Default	The name that is delivered by the SNC library by default.

Property	***sncsso***=<should SSO (single sign-on) be used>
Description	Specifies whether single sign-on should be used.
Required	No
Values	0: No SSO; user and password are used for authentication; encryption is used for communication. 1: The SNC identity is used for authentication; encryption is used for communication
Default	1

Property	***sncpartnername***=<SNC name on server side>
Description	SNC name on server side (like p:CN=xxx, OU=yyy, O=zzz, C=DE)
Required	This parameter is only used and needed if logon is done via the application server.
Values	Character string
Default	---

Property	***sncqop***=<quality of protection>
Description	Protection level for the communication.
Required	No; it is only used if ***sncsso***=1.
Values	1 : Authentification protection 2 : Integrity protection 3 : Privacy protection 8 : Apply default protection 9 : Maximum protection
Default	3

5.1.1.8 Logon Information (Call Type HTTPStateful)

Property	***https***=<should HTTPS be used>
Description	Specifies whether HTTP or HTTPS should be used
Required	No
Values	0: HTTP 1: HTTPS
Default	0

Property	***httpserverhost***=<http server host>

Description	Host of the http server (as needed for the URL).
Required	Only if ***calltype=HTTPStateful.***
Values	Character string
Default	---

Property	***httpserverport***=<port number>
Description	Port number of the http server. Make sure that the port number is specified in your system with the profile parameter ***icm/server_port_x***.
Required	Only if ***calltype=HTTPStateful.***
Values	Digits representing the port number
Default	---

The following properties are only used if ***https*** = 1.

The following authentication types are supported for HTTPS:

- Server authentication
  Trust store is used (if not set, JRE default is used).
  - With user/password
  - Without user/password
    User/password has to be set on server side in transaction SICF for service 'hatool'.
- Mutual authentication
  Trust store and keystore are used.

Property	***httpstruststore***=<file name of trust store>
Description	The trust store is used to store the root certifcate of the server to allow server authentication. Alternatively it could be provided as a Java property on the command line: -Djavax.net.ssl.trustStore=<file> Setting both the Java and the HA Test Tool property results in a 'Parameter declared multiple times' error.
Required	No
Values	Character string
Default	<JRE_HOME>/lib/security/cacerts

Property	***httpstruststorepassword***=<password>
Description	Password of the trust store. Alternatively it could be provided as a Java property on the command line: -Djavax.net.ssl.trustStorePassword=<password> Setting both the Java and the HA Test Tool property results in a 'Parameter declared multiple times' error.
Required	No
Values	Character string
Default	Default password of <JRE_HOME>/lib/security/cacerts

Property	***httpskeystore***=<file name of keystore>

Description	The keystore is used to store the client certifcate in PKCS12 format. It is only needed if mutual authentication should be done. Alternatively the name of the store could be provided as a Java property on the command line: -Djavax.net.ssl.keyStore=<file> Setting both the Java and the HA Test Tool property results in a 'Parameter declared multiple times' error.
Required	No
Values	Character string
Default	---

Property	***httpskeystorepassword***=<password>
Description	Password of the key store. Alternatively it could be provided as a Java property on the command line: -Djavax.net.ssl.keyStorePassword=<password> Setting both the Java and the HA Test Tool property results in a 'Parameter declared multiple times' error.
Required	No
Values	Character string
Default	---

5.1.1.9 Communication Traces

Communication traces should only be turned on to investigate communication errors.

Traces for ***calltype=RFCStateful***:

Property	***jcotrace***=<trace level of JCo trace>
Description	The trace level of the JCo trace is specified. The JCo trace is written to the subdirectory jcotrace of the HA Test Tool base directory. File name: JCO<YYYYMMDD_HHMMSS...>.trc.
Required	No
Values	0 – 10 0: Trace off 1 – 9: Trace on; see javadoc of JCo.setTrace()
Default	0

Property	***rfctrace***=< RFC trace on/off>
Description	The RFC trace is written to the subdirectory jcotrace of the HA Test Tool base directory. File name: jco_rfc....trc.
Required	No
Values	0: Trace off 1: Trace on
Default	0

Property	***cpictrace***=< trace level of CPIC trace>
Description	The trace level of the CPIC trace is specified. The CPIC trace is written to the current working directory. File name: CPIC<YYMMDD_HHMMSS...>.trc.

Required	No
Values	0 – 3 0: Trace off 1 - 3: Trace on; 3: most verbose trace level
Default	0

There are no trace properties for ***calltype=HTTPStateful***. If HTTP errors occur, an HTTP trace is written together with the stack trace information to the log file for the main thread (see chapter 6.2.2) and to the error files for the client threads (see chapter 6.3.4).

5.1.1.10 HA Events

An overview of the event handling is given in chapter 9.2. The details concerning the properties to specify one or several events are shown below.

One HA Event:

Property	***eventstarthighload***=<starting point relative to beginning of high load phase (in seconds)>
Description	The clock is ticking when the high load phase starts or, in other words, when the period specified by the property ***duration*** begins. The HA event has to start within the high load phase and the system should return to normal behavior (concerning availability, response times, throughput) during that phase again.
Required	No
Values	Number (not negative) The following condition should be fulfilled concerning the properties: ***duration*** > ***eventstarthighload*** + period in which HA event effects the system behavior
Default	---

Property	***eventname***=<name of HA event>
Description	Name of HA event in human readable form (no technical name).
Required	No
Values	Character string
Default	---

Property	***eventcall***=<call of HA event>
Description	The event call can be a script or a program that has to be specified with all parameters.
Required	No
Values	Character string The corresponding file has to be located in the subdirectory ***event*** of the HA Test Tool base directory. Therefore, the name must not contain a directory path. As of version 2.13, you can specify variable call parameters. You can use either user-defined properties or test properties (except event properties (chapter 5.1.1.10), file collection properties (chapter 5.1.1.1), and password properties). User-defined properties are enclosed in <(…)>. Test properties are enclosed in <…>. The values of user-defined properties are specified by the property ***eventcallparameters*** (see below). If a test property is used but not set, its default is used. If a default does not exist, the program aborts.

	Example: eventcall=script.sh 20 <systemid> <(myparam)>
Default	---

Property	***eventcallparameters***=<values of user-defined variables for HA event call>
Description	This property was introduced with version 2.13. The values of the user-defined variables of the property ***eventcall*** are specified. The right-hand side must not contain white space characters. You must use the following format: eventcallparameters=<param1>:<value1>[;<param2>…] Example: eventcallparameters=myparam1:100;myparam2:force If the property is used on the command line, you should consider the special character handling of the shell. Especially ';' is used if two eventcall parameters are specified. In such a case, you must quote the parameter value. Example: Windows: eventcallparameters="myparam1:myvalue1>;myparam2:myvalue2" Linux/Unix: eventcallparameters='myparam1:myvalue1>;myparam2:myvalue2'
Required	No
Values	Character string
Default	---

Property	***eventdefinitionfile***=<definition file of HA event>
Description	Event properties file of the event. The same kind of HA event can be used for various tests. Therefore, you can specify its name (***eventname***) and the call (***eventcall***) in a separate file instead of setting them directly in the property file of the test. Since the starting point (property ***eventstarthighload***) belongs to the test, this property must not be set in the file. As an example, the file ***haEventTemplate.properties*** is included in the HA Test Tool package.
Required	No
Values	Character string The event properties file has to be located in subdirectory ***config*** of the HA Test Tool base directory. Therefore the name must not contain a directory path.
Default	---

Several HA Events:

If several HA events should be triggered during a test run, they have to be specified by event definition files. The events have to be consecutively numbered starting with 1. It is possible to specify isolated events as well as event chains. Such chains can be build via property ***eventstartprevious*** (see below) that specifies the starting point of the next event in relation to the end of the previous event.

Property	***eventstartprevious.n***=<starting point in seconds in relation to the end of the previous event >
Description	The clock is ticking when the previous event call (event n-1 in this example) has ended with exit code 0. If the exit code of the previous event ≠ 0, the event is not executed. The HA event has to start within the high load phase and the system should return to normal behavior (concerning availability, response times, throughput) during that phase again.
Required	No
Values	Number (not negative)

Default	---

For the other properties needed, see the description of ***eventstarthighload*** and ***eventdefinitionfile*** above.

Several events can be defined in the following way:

- ***eventstarthighload.1***=<starting point of the first event in relation to the beginning of the high load phase (in seconds)>
  eventdefinitionfile.1=<definition file of the first HA event>
  -
- ***eventstarthighload.2***=<starting point of the second event in relation to the beginning of the high load phase (in seconds)>
  Event 2 is independent of event 1.
  or
- ***eventstartprevious.2***=<starting point of the second event in relation to the end of the previous event (in seconds)>

  eventdefinitionfile.2=<definition file of the second HA event>
  -
  … (more events)

If the property ***eventcall*** of ***eventdefinitionfile.n*** contains user-defined variables, the values have to be specified by the property ***eventcallparameters.n***. This property must be outside of the event definition file.

Example:

eventdefinitionfile.2 contains ***eventcall***=kill.bat <(killmode)> <(timeout)> <systemid> outputfile.log
eventcallparameters.2=timeout:10;killmode:force (is specified outside of the event definition file)
systemid=SID

➔ The following call is done: kill.bat force 10 SID outputfile.log

5.1.1.11 Result Check

As of version 2.02, you can provide a description of the expected result of the test. At the end of the test run, the test outcome is compared against the expected result. The check is documented in the files ***loadtest.summary*** and ***loadtest.xml*** (chapter 6.2). The exit code is adapted accordingly.

Property	***checkfile***=<definition file for the expected result>
Description	In the file, the expected result of the test run is specified. As an example, the file ***haCheckTemplate.xml*** is included in the HA Test Tool package. Here and in chapter 5.1.2 (Specification of the Expected Test Result), you find details about the format of the check file.
Required	No
Values	Character string It has to be located in the subdirectory ***check*** of the HA Test Tool base directory. Therefore, the name must not contain a directory path.
Default	---

5.1.1.12 HTML Output

As of version 2.12, a graphical summary of the test run is written to the file ***loadtest.html***. To display the file in a browser, the graphics library ***d3.min.js*** has to be available. During the test run, the location of the graphics library has to be known because the path name is generated into ***loadtest.summary.***

Property	***graphicslibrary***=<full path name of the graphics library for the HTML output>

Description	The graphics library ***d3.min.js*** is needed to display the file ***loadtest.html*** in a web browser. ***d3.min.js*** is an open source library. You can download it from ***https://d3js.org/*** as part of ***d3.zip***. ***loadtest.html*** is generated even if the graphics library does not exist (a warning is then written to ***loadtest.log***).
Required	No
Values	Character string You have to specify the full path name of the graphics library ***d3.min.js***.
Default	<HA Test Tool base directory>/bin/d3.min.js

5.1.2 Specification of the Expected Test Result

Triggering HA events during a test run can cause a disturbance of the server system. Responses of calls can be delayed or errors that indicate the unavailability of the server or server components can occur.

As of version 2.02 of the HA Test Tool, you can provide a description of the expected result of a test run to the HA Test Tool. At the end of the test run, the test outcome is compared against the expected result. The check is documented in the files ***loadtest.summary, loadtest.html*** and ***loadtest.xml*** (chapter 6.2). The exit code is adapted accordingly.

The specification of the expected result has to be provided as an XML file to the tool (see property ***checkfile*** in chapter 5.1.1). The file has to be located in the subdirectory ***check*** of the HA Test Tool base directory. The correct format is checked by the file ***xmlresult.dtd*** that is provided by the HA Test Tool package (chapters 4.1 and 4.2).

In the check file, only the server behavior that is expected to occur after HA events can be specified, that is:

- Unavailability of the server or server components
- Long response times
- Errors of certain categories indicating unavailability

Other problems like HA event errors, verification errors or error categories that must not occur (like fatal errors) cannot be specified in the check file. They should not occur as results of a test run.

In the following we describe what a check file should look like. As an example, the file ***haCheckTemplate.xml*** is provided by the HA Test Tool package (chapters 4.1 and 4.2).

Frame Mandatory part; the other parts described in the table below are optional. The corresponding check is performed only if the tag is specified. ***xmlresult.dtd*** is provided by the HA Test Tool package and must not be changed.	*<?xml version="1.0" encoding="UTF-8" standalone="no"?>* *<!DOCTYPE XmlResult SYSTEM "xmlresult.dtd" >* *<XmlResult>* *… <!—content described in the table below -->* *</XmlResult>*
Unavailability There are different ways to specify that periods of unavailability are allowed: - Allow any period of unavailability of any time - Allow periods of unavailability up to a certain number (n)	*<UNAVAILABILITY />* *<UNAVAILABILITY>* *<number_of_periods>n</number_of_periods>* *</UNAVAILABILITY>*

- Allow periods of unavailability up to a certain duration for each period (in seconds) - Allow periods of unavailability up to a certain number and duration	*<UNAVAILABILITY>* *<duration>s</duration>* *</UNAVAILABILITY>* *<UNAVAILABILITY>* *<number_of_periods>n</number_of_periods>* *<duration>s</duration>* *</UNAVAILABILITY>*
Long response times There are different ways to specify that periods of long response times are allowed: - Allow any period of long response times of any time - Further checks concerning the number of periods and duration are the same as with unavailability	*<LONG_RESPONSETIMES />* Format: see <UNAVAILABILITY> above
Errors - Allow errors of any category (We do not recommend this because this allows errors that should not be caused by HA events (like fatal errors).) The following error categories can be allowed. Any combination of them can be specified. - Errors that indicate the unavailability of server components - Errors that indicate that the server cannot be reached: o Communication errors o Error during logon o Errors during logoff o Call repetition error (*)	*<ERRORS />* *<ERRORS>* *<app_notavail_err />* *<comm_err />* *<login_err />* *<logoff_err />* *<app_repeat_err />* *</ERRORS>*

(*): A call repetition error (app_repeat_err) occurs if a call is issued only once by the HA Test Tool client, but is executed several times by the server. This error should never occur without an HA event. If the error occurs, when the HA event kills the WebDispatcher or the ICMan and HTTP is used as protocol, the reason might be an internal retry of the JRE HTTP client. In such a case, APP_REPEAT_ERR can be specified as an expected result. Otherwise the error shows that the system does not react correctly to this type of event.

5.2. Starting the Test Run

On server side, the system must be up and running.

On client side, go to the directory of ***haTestTool.jar*** (subdirectory ***bin*** of the HA Test Tool base directory and start the test driver with the following command:

java -cp haTestTool.jar;<jcoDirectory>\sapjco3.jar com.sap.test.haload.ClientDriver param1=value1 …

On Unix/Linux, use ':' instead of ';' to separate the jar files in the class path. And, of course, '/' instead of '\' in the directory paths.

The parameters are described in detail in chapter 5.1.1. Make sure that there are no spaces in between a key=value pair.

To avoid specifying all single parameters in the command line, we recommend to collect them in a properties file that has to be located in subdirectory ***config*** of the HA Test Tool base directory. Call the test driver with the following command:

java -cp haTestTool.jar;<jco.jar file> com.sap.test.haload.ClientDriver file=<properties file>

The installation contains the files ***haTemplate.properties*** and ***haQuickStartTemplate.properties***, which can be a starting point for this purpose.

If properties are set in the command line and in the property file, the value in the command line takes precedence.

6. Analysis of Results

This chapter describes the structure and the contents of the data that is generated by the HA Test Tool.

6.1. Structure of Output Data

The data is located in the subdirectory ***output*** of the HA Test Tool base directory. To separate the data of different test runs, a subdirectory structure is created that reflects the values of ***testrunid, clienthost, calltype*** and ***testclass.*** This means that the output of a test run is put in directory ***…HATool/output/<testrunid>/<clienthost>/<calltype>/<testclass>***.

This directory contains the following information:

- ***loadtest.log*** Log file of the test run
- ***loadtest.summary*** Summary and analysis of the test run in text format
- ***loadtest.xml*** Summary and analysis of the test run in XML format
- ***loadtest.html*** Graphical summary and analysis of the test run in HTML format
- ***loadtest.perf.csv*** .csv file of throughput and response time data
- ***client*** Directory with information about the simulated clients
  - ***user00001.res*** Response file of client 1
  - ***user00001.summary*** Summary and analysis of the test run of client 1
  - ***user00001.err*** Erroneous calls of client 1 (overview)
  - ***user00001.elg*** Erroneous calls of client 1 (stack trace)

  - ***user00002.res*** Response file of client 2
  - ***…***
- ***event*** Directory with information about the triggered events
  - ***event001.log*** Log file of event 1 written by the test tool
  - ***event001.out*** Output of event 1 written by the event

  - ***event002.log*** Log file of event 2 written by the test tool
  - ***…***

6.2. Main Output Data of the Test Run

6.2.1 Summary File (Text Format)

loadtest.summary is the most important source to analyze the result of a test run. It contains the following information:

- HA Test Tool version
- Start time of the test run
- Most important properties used for the test run
- HA events triggered during the test run
- Availability and responsiveness of the system
  - Information about periods of erroneous calls indicating the unavailability of the system or system components
  - Information about long response times of successful calls
- Verification information; indicates whether the test was semantically correct
- Error overview
- Data about performance and throughput in the following granularities:
  - Complete high load phase
  - Every tact (The high load phase is split into pieces, called tacts, to allow a better analysis of throughput and response time during the run.)
  - Only successful calls are taken into account
- Checks against the expected test result

- Exit code of the test run

Example of ***loadtest.summary*** (DB was restarted during the test):

```
LOADTEST v2.14
Started at              2017-12-21 13:25:35.439

Test RunID:             214_c.DbRstFil
Test class:             CL_HA_DB_LOAD
Call type:              RFCStateful
Destination:            xxxxxxxx
Output directory:       C:\HATool\output\214_c.DbRstFil\xxxxxxxx\RFCStateful\CL_HA_DB_LOAD
checkfile:              C:\HATool\check\db.restart.xml
Number of clients:      10
Steps per loop:         5
Loops per session:      1
Think time(s):          5
Subtest Parameters:     ItemsPerOrder:50
Test duration(s):       300
Duration after events: 60

Internal ID of subtest: 3769

High load phase started at: 2017-12-21 13:26:26.735
High load phase ended at:   2017-12-21 13:28:32.888
2017-12-21 13:28:52.531 Test finished


H A   E V E N T S
-----------------

Event 1: DB Restart
  Status: EXECUTED     (Exit code: 0)
  Duration:   2017-12-21 13:26:51.772 - 2017-12-21 13:27:32.886  (41,114s)


A V A I L A B I L I T Y and R E S P O N S I V E N E S S
-------------------------------------------------------

System ok:        2017-12-21 13:25:36.783 - 2017-12-21 13:26:58.225
System disturbed: 2017-12-21 13:26:58.226 - 2017-12-21 13:27:33.807  (35,581s)
System ok:        2017-12-21 13:27:33.808 - 2017-12-21 13:28:50.347

The periods of disturbance of the system are caused either by erroneous calls
that indicate that the system or parts of the system are not available or
by calls that exceed the threshold for long response times suggesting
that long wait times inside the system occurred during the calls.
For more details, see the following sections:
- Disturbance, indicated by erroneous calls
- Disturbance, indicated by long response times


D I S T U R B A N C E, indicated by E R R O N E O U S  C A L L S  (U N A V A I L A B I L I T Y)
-----------------------------------------------------------------------------------------------

Severe errors in part of the calls: 2017-12-21 13:26:58.226 - 2017-12-21 13:27:19.246  (21,020s)

The errors that indicate periods of restricted or no availability are caused by following reasons:

Component(s) on the server could not be reached.
This is indicated by the error code APP_NOTAVAIL_ERR.
For more details, search for APP_NOTAVAIL_ERR in the client output files


D I S T U R B A N C E, indicated by L O N G  R E S P O N S E  T I M E S
-----------------------------------------------------------------------

Threshold for long response times = 10 seconds

Partially long response times: 2017-12-21 13:26:59.238 - 2017-12-21 13:27:33.807  (34,569s)
```

… (Explanation of long response times)

```
V E R I F I C A T I O N
-----------------------

Verification of server data after the test run showed no errors

  Verification result of frame test class: CL_HA_TESTFRAME_DB

    OK: All DB tables have the correct number of rows with int_runid 0.

  Verification result of test class: CL_HA_DB_LOAD

    DB table size check of int_runid 3769
      OK. Number of rows in DB table HALOAD_DB_MAT = 1000
      OK. Number of rows in DB table HALOAD_DB_CUST = 1000
      OK. Number of rows in DB table HALOAD_DB_ORDER = 60
      OK. Number of rows in DB table HALOAD_DB_ORDITM = 1500

    Result of checks of int_runid 3769
      No check errors occurred

    Verification summary of test class: CL_HA_DB_LOAD
      OK: All DB tables have the correct number of rows for checked int_runids.
      OK: No check errors occurred during the test.


E R R O R   O V E R V I E W
---------------------------

Number of erroneous calls:    3

Errors during execution of steps
  Number of application errors (APP_NOTAVAIL_ERR): 3
```

… (Explanation of error categories)

```
P E R F O R M A N C E and T H R O U G H P U T
---------------------------------------------

Time range:  2017-12-21 13:26:26.735 - 2017-12-21 13:28:32.888
Number of steps:    183
Number of logins:   40
Steps per second:   1,5
Avg. response time in ms:          1444,7
Avg. elapsed time(Server) in ms:   33,7
Avg. response time of login in ms: 16,0
  Step 1
    Avg. response time in ms:          2961,2
    Avg. elapsed time(Server) in ms:   36,8
  Step 2
    Avg. response time in ms:          881,8
    Avg. elapsed time(Server) in ms:   31,9
  Step 3
    Avg. response time in ms:          26,5
    Avg. elapsed time(Server) in ms:   23,5
  Step 4
    Avg. response time in ms:          26,6
    Avg. elapsed time(Server) in ms:   23,5
  Step 5
    Avg. response time in ms:          2788,7
    Avg. elapsed time(Server) in ms:   49,0
```

```
Tact duration = 10 seconds

  Number          Start             Number    Steps       Avg.        Avg.        Avg.
    of             of                of       per      Response     Server      Login
   Tact           Tact              Steps    Second    Time (ms)   Time (ms)   Time (ms)
-------------------------------------------------------------------------------------------
      1   2017-12-21 13:26:26.735      20       2,0         17,6        14,0        14,4
      2   2017-12-21 13:26:36.737      20       2,0         27,3        23,8         0,0
      3   2017-12-21 13:26:46.737      20       2,0         23,0        19,8        14,8
      4   2017-12-21 13:26:56.738       2       0,2         39,0        35,5        13,0
      5   2017-12-21 13:27:06.739       0       0,0          0,0         0,0         0,0
      6   2017-12-21 13:27:16.739       0       0,0          0,0         0,0         0,0
      7   2017-12-21 13:27:26.740      10       1,0      26032,9       268,9        30,2
      8   2017-12-21 13:27:36.740      19       1,9         27,6        24,3         0,0
      9   2017-12-21 13:27:46.741      20       2,0         22,0        19,1        18,8
     10   2017-12-21 13:27:56.741      20       2,0         20,3        16,6        13,4
     11   2017-12-21 13:28:06.742      20       2,0         28,2        25,0        13,0
     12   2017-12-21 13:28:16.744      20       2,0         17,4        14,2        12,8
     13   2017-12-21 13:28:26.744      12       2,0         28,0        24,4         0,0

Duration of last tact: 6,144s


C H E C K against E X P E C T E D   R E S U L T
-----------------------------------------------

Check result against C:\HATool\check\db.restart.xml

Check for unavailability
  OK: 1 period(s) of unavailability occurred; up to 1 expected.
  OK: Duration of period 1: 21,020 seconds; up to 600 seconds acceptable.

  Result:
    OK: Period(s) of unavailability behaved as expected.

Check for long response times
  OK: 1 period(s) of long response times occurred; up to 1 expected.
  OK: Duration of period 1: 34,569 seconds; up to 600 seconds acceptable.

  Result:
    OK: Period(s) of long response times behaved as expected.

Check for errors
  OK: Errors of category APP_NOTAVAIL_ERR expected.

  Result:
    OK: Only expected errors occurred.

Final result:
  OK: Outcome of test run was as expected.


E X I T   C O D E
-----------------

Exit code based on the analysis of the test run: 0 (OK)
```

If the database is restarted during the test, there will be calls returning errors because of the unavailability of the database, while other calls will show long response times because they will be waiting for a reconnect to the database. Therefore, the test run above behaves as expected.

The summary file contains not only raw data, but also a description of it. Since the output for calls with long response times might be difficult to understand, see a more detailed explanation:

There could be periods with no responses and periods with partially long responses.

A period of ‘No response’ means the following:

- All calls to the server issued in this period last at least as long as specified by the threshold.
- The first response of the server to these calls occurs at the end of the period. This means that during this period the server is not responsive.

A period of ‘Partially long response times’ may have following reasons:

- At least one but not all client threads experience response times that are greater or equal than the threshold.
- If the period occurs directly after a period of no response, the reason might also be that the remaining slow calls finished. If the period is long, it means that the server needs quite some time to return to acceptable response times.
- If the period occurs directly before a period of no response, it is very likely that some of the client threads issued calls that do not use the component that is not responsive, whereas other calls are waiting for a reaction of this component. The time of the period with partially long response times plus the period of no response could, for example, indicate how long a component is not responsive because of a switchover.

6.2.2 Log File

The file ***loadtest.log*** contains the log data of the main thread of the test run. You find information such as:

- HA Test Tool version
- Start time of the test run
- Properties used for the test run
- Server version of the HA Test Tool
- Description of test class
- Internal ID of subtest run
- Important status messages
- Errors thrown by the main thread
- Exit code based on the analysis of the test run

Example of ***loadtest.log***:

```
LOADTEST v2.14
Started at              2017-12-21 13:25:35.439

Test Properties:
clienthost:             xxxxxxxx
testrunid:              214_c.DbRstFil
testclass:              CL_HA_DB_LOAD
calltype:               RFCStateful
outputdir:              C:\HATool\output\214_c.DbRstFil\xxxxxxxx\RFCStateful\CL_HA_DB_LOAD
systemclient:           001
language:               en
sncmode:                0
user:                   HA_TEST
msgserverhost:          xxxxxxxx
msgserverport:          3600
logongroup:             SPACE
clientnum:              10
clientdelay:            1
stepnum:                5
loopspersession         1
duration:               300
durationafterevents:    60
thinktime:              5
errorthinktimelow:      3
errorthinktimehigh:     8
resettestdata:          1
testparameters:         not set
subtestparameters:      ItemsPerOrder:50
tactduration:           10
noresponsethreshold:    10
graphicslibrary:        C:\HATool\bin\d3.min.js
checkfile:              C:\HATool\check\db.restart.xml
HA Event 1
  eventname:            DB Restart
  eventstarthighload:   25
  eventdefinitionfile:  C:\HATool\config\event.db.restart.txt
  eventcall:            C:\HATool\event\db.restart.bat

Server version:         v2.13
```

```
Description of test class CL_HA_DB_LOAD:

<Short text>
Database load test class

<Long text>
CL_HA_DB_LOAD generates configurable load on the database. Additionally, it
performs data consistency checks during and at the end of the test run.
CL_HA_DB_LOAD extends the frame test class CL_HA_TESTFRAME_DB.

<Resources>
Following database tables are used by CL_HA_DB_LOAD:
  - HALOAD_DB_MAT:    material table
  - HALOAD_DB_CUST:   customer table
  - HALOAD_DB_ORDER:  order table
  - HALOAD_DB_ORDITM: order item table
  - HALOAD_DB_CHECK:  table for check results

<Initial state>
The first column of each of these tables is called INT_RUNID. For the initial
data, provided by CL_HA_TESTFRAME_DB, it is set to 0.
The initial data of the tables HALOAD_DB_MAT and HALOAD_DB_CUST is copied
within the tables, setting INT_RUNID to the internal ID of the subtest run. For
the tables HALOAD_DB_ORDER, HALOAD_DB_ORDITM and HALOAD_DB_CHECK there is
initially no content for the subtest run. Because the internal ID is unique for
subtest run, the data used resp. created by such a run is unique too.

<Call sequence>
The test class implements 5 different steps:
  - Insert:   Inserts one order with order items
  - Update:   Updates this order and the order items
  - Delete:   Deletes half of the order items and updates the order
  - Rollback: Inserts the same order again (-> duplicate key error), inserts
              a new order and executes a rollback
  - Check     Executes various data consistency checks at the end of the call
              sequence; additionally checks are also done within each step
According to the property stepnum that could have a value between 2 and 5
following steps are executed:
  - stepnum = 2: Insert + Check
  - stepnum = 3: Insert + Update + Check
  - stepnum = 4: Insert + Update + Delete + Check
  - stepnum = 5: Insert + Update + Delete + Rollback + Check


<Final state>
The tables HALOAD_DB_MAT and HALOAD_DB_CUST are not changed. HALOAD_DB_ORDER
and HALOAD_DB_ORDITM contain the orders and order items that are inserted and
modified during the subtest run. Table HALOAD_DB_CHECK contains the check
results. For all tables the column INT_RUNID contains the ID of the subtest
run.

<Checks>
During the subtest run various consistency checks are done concerning the data
in the DB tables. At the end it is checked, whether all tables contain the
expected number of rows. Only data generated and used for this specific subtest
run is checked.

<Configuration>
Besides different values for the number of users and the number of steps, the
number of order items per order could be specified by the property
subtestparameters. A value between 1 and 10000 is allowed. Default is 10. Take
care that with a big number of items per order a huge load on the database and
much data are generated.
Syntax: subtestparameters=ItemsPerOrder:nnn

<Attributes>
Max. number of steps:       5
Min. number of steps:       2
Reset of test class needed: 0
```

```
Internal ID of subtest: 3769

2017-12-21 13:25:46.733 All client threads started
2017-12-21 13:26:26.735 High load phase started
2017-12-21 13:26:51.772 Event 1 started (DB Restart)
2017-12-21 13:27:18.286 Severe error when calling the server
2017-12-21 13:27:32.886 Event 1 finished; exit code 0 (DB Restart)
2017-12-21 13:27:33.809 All client threads continue with their work
2017-12-21 13:28:32.888 High load phase ended
2017-12-21 13:28:50.363 All client threads finished their work
2017-12-21 13:28:52.531 Test finished

Exit code based on the analysis of the test run: 0 (OK)
```

The exit code is returned by the client driver and allows a rough automated analysis of the test run. There are the following exit codes:

OK	0
Long response times	1
Erroneous calls	2
Unavailability	4
Verification error	8
HA Event problem	16
Fatal error	32

The exit code reflects the test analysis shown in more detail in ***loadtest.summary***. If several incidents happened, the corresponding codes are just added. For example, the verification showed problems (8), the system was temporary not available (4), and there were erroneous calls (2). This results in exit code 14 (8 + 4 + 2).

Starting with version 2.11, the exit code

No execution	-1

was introduced. It is set if for ***specialaction=CleanAdminData*** (see chapter 5.1.1.2) the cleanup of the administrative data of an active test was not confirmed.

6.2.3 Summary File (XML Format)

Starting with version 2.02, a summary file in XML format is generated to ease the automatic processing of the test result. Basically, this file contains the property information of the log file (chapter 6.2.2) and the test outcome of the summary file (chapter 6.2.1).

The format of the file is described in the table below (real values are used in the examples to describe the content of tags and the tag attributes):

Frame Mandatory part; it is filled in any case, even if only a special action like ***CleanAdminData*** or ***ResetTestData*** (see chapter 5.1.1) is performed. Only those properties that are relevant for the test are listed. They are identical with the ones shown in the log file (chapter 6.2.2), except the HA event related properties: they are shown in the tag ***<HAEVENTS>***.	*<?xml version="1.0" encoding="UTF-8"?>* *<XmlSummary>* *<LOADTEST version="v2.11"* *starttime="2015-01-21 18:23:59.018">* *<PROPERTIES>* *…* *</PROPERTIES>* *…* *<EXITCODE>0</EXITCODE>*

The property tag names are identical with the property names listed in chapter 5.1.1. Additionally, the tag name ***<outputdir>*** is used to describe the directory of the test output. All tags described below are between the ***<PROPERTY>*** and the ***<EXITCODE>*** tag.	*</LOADTEST>* *</XmlSummary>*
Information written during the test run - Server version of the HA Test Tool - Internal ID of subtest run - Start of high load phase - End of high load phase - Errors that occurred in the course of the test run (not during processing of calls from the client to the server)	*<server_version>v2.13</server_version>* *<internal_subtestID>4711</internal_subtestID>* *<highload_start>2015-01-21 17:59:35.809</highload_start>* *<highload_end>2015-01-21 18:01:35.813</highload_end>* *<errormessage> message text </errormessage>*
HA events If there is no HA event, the tag is not shown. - Number of events For each event, the following information is written: - Event number and event properties (see chapter 5.1.1) as attributes (the prefix 'event' is omitted). File names are shown with their full path name. - Outcome of the event o Status (values: ***NOTSTARTED, STARTED, EXECUTED, ABORTED***) o Result code o Duration (in seconds) o Start time o End time	*<HAEVENTS>* *<number_of_events>2</number_of_events>* *<event num="1" name="DB Restart" starthighload="25" call="C:\HATool\event\db.restart.bat" definitionfile="C:\HATool\config\event.db.restart.txt">* *<status>EXECUTED</status>* *<result>0</result>* *<duration>42,319</duration>* *<starttime>2015-01-21 17:54:24.700</starttime>* *<endtime>2015-01-21 17:55:07.019</endtime>* *</event>* *<event num="2" …* *</event>* *</HAEVENTS>*
Disturbance Periods in which the system was disturbed are listed. If there are no such periods, tag ***<number_of_periods>*** = 0 is shown. Otherwise, first a general overview of the disturbed periods is given, and then the details are listed so that you can see how they are made up of periods of unavailability and long response times. If there is no such period, the corresponding tag is omitted. There is a difference between the data in the summary file (chapter 6.2.1) and the XML file: In general, it is differentiated between periods of unavailability or long response times that were experienced by all clients (tag **<degree>**	*<DISTURBANCE>* *<number_of_periods>1</number_of_periods>* *<period num="1">* *<duration>36,661</duration>* *<starttime>2015-01-21 18:06:16.928</starttime>* *<endtime>2015-01-21 18:06:53.589</endtime>* *</period>* *<UNAVAILABILITY>* *<number_of_periods>1</number_of_periods>* *<period num="1">* *<degree>partial</degree>* *<duration>20,954</duration>* *<starttime>2015-01-21 18:06:16.928</starttime>*

= full) or by only part of the clients (tag ***<degree>* = partial**). If there is no time gap between such periods, they are merged into one period in the XML file (tag ***<degree>* = *mixed***), and the duration, start and end time are adapted accordingly. In the summary file (chapter 6.2.1), they are shown as separate periods.	*<endtime>2015-01-21 18:06:37.882</endtime>* *</period>* *</UNAVAILABILITY>* *<LONG_RESPONSETIMES>* *<number_of_periods>1</number_of_periods>* *<period num="1">* *<degree>partial</degree>* *<duration>35,674</duration>* *<starttime>2015-01-21 18:06:17.915</starttime>* *<endtime>2015-01-21 18:06:53.589</endtime>* *</period>* *</LONG_RESPONSETIMES>* *</DISTURBANCE>*
Verification Only the result of the verification is shown (***ok, error***), not the details. You can find the details in the corresponding section in the summary file (chapter 6.2.1).	*<VERIFICATION>* *<result>ok</result>* *</VERIFICATION>*
Errors The complete number of erroneous calls is shown. If there are no errors, tag ***<number_of_errors>*** = 0 is shown. For each affected error category, the number of erroneous calls is listed. If there are no errors, the error category is not shown. The example on the right lists all possible categories. In the text of the summary file (chapter 6.2.1) you can find an explanation of the error categories.	*<ERRORS>* *<number_of_errors>16</number_of_errors>* *<fatal_err>1</fatal_err>* *<comm_err>1</comm_err>* *<login_err>4</login_err>* *<logoff_err>1</logoff_err>* *<app_inconsistent_err>1</app_inconsistent_err>* *<app_notavail_err>3</app_notavail_err>* *<app_repeat_err>1</app_repeat_err>* *<app_err>1</app_err>* *<conv_err>2</conv_err>* *<other_err>1</other_err>* *</ERRORS>*
Performance and throughput The tags do not vary. They express 1:1 the contents of the corresponding section of the summary file (chapter 6.2.1). For the tags **<step>** and **<tact>,** the content is listed via attributes.	*<PERFORMANCEandTHROUGHPUT>* *<number_of_steps>183</number_of_steps>* *<number_of_logins>39</number_of_logins>* *<steps_per_second>1,5</steps_per_second>* *<avg_responsetime>1498,6</avg_responsetime>* *<avg_servertime>34,7</avg_servertime>* *<avg_logintime>23,3</avg_logintime>* *<step num="1" avg_responsetime="1801,4" avg_servertime="16,5"/>* *<step num="2" …* *<tact num="1" starttime="2015-01-21 18:05:44.601" number_of_steps="20" steps_per_second="2,0" avg_responsetime="19,9" avg_servertime="17,0" avg_logintime="15,8"/>* *<tact num="2" …* *<duration_of_last_tact>6,350</duration_of_last_tact>* *</PERFORMANCEandTHROUGHPUT>*
Check of expected result	*<CHECK_RESULTS>* *<UNAVAILABILITY>*

This tag is only used if the property checkfile (chapter 5.1.1) has been set. The outcome of the single checks is shown by the values ***ok*** and ***error***. At the end of all checks, the result is summarized by the tag ***<FINAL_RESULT>***. In addition to the values ***ok*** and ***error,*** the value ***ok_but*** is set if all checks are ok but there are other errors in the test run. In the example at the right the tool checked for the maximum number and maximum duration of periods of unavailability. Periods of long response times of any number and duration are allowed as well as errors of the category APP_NOTAVAIL_ERR. All checks were ok, but you can see that the test run still has a problem: the verification has failed. If there are errors of other categories than APP_NOTAVAIL_ERR, these categories are listed with the value ***error***.	*<number_of_periods>ok</number_of_periods>* *<period num="1">* *<duration>ok</duration>* *</period>* *<period num="2">* *<duration>ok</duration>* *</period>* *<result>ok</result>* *</UNAVAILABILITY>* *<LONG_RESPONSETIMES>* *<result>ok</result>* *</LONG_RESPONSETIMES>* *<ERRORS>* *<app_notavail_err>ok</app_notavail_err>* *<result>ok</result>* *</ERRORS>* *<FINAL_RESULT>* *<result>ok_but* *<VERIFICATION />* *</result>* *</FINAL_RESULT>* *</CHECK_RESULTS>*

6.2.4 Graphical Summary File (HTML Format)

Starting with version 2.12, a summary file in HTML format is generated to provide a graphical view of the test result (***loadtest.html***). Basically, this file contains the test outcome of the summary file described in chapter 6.2.1 in graphical form and the property information of the log file described in chapter 6.2.2.

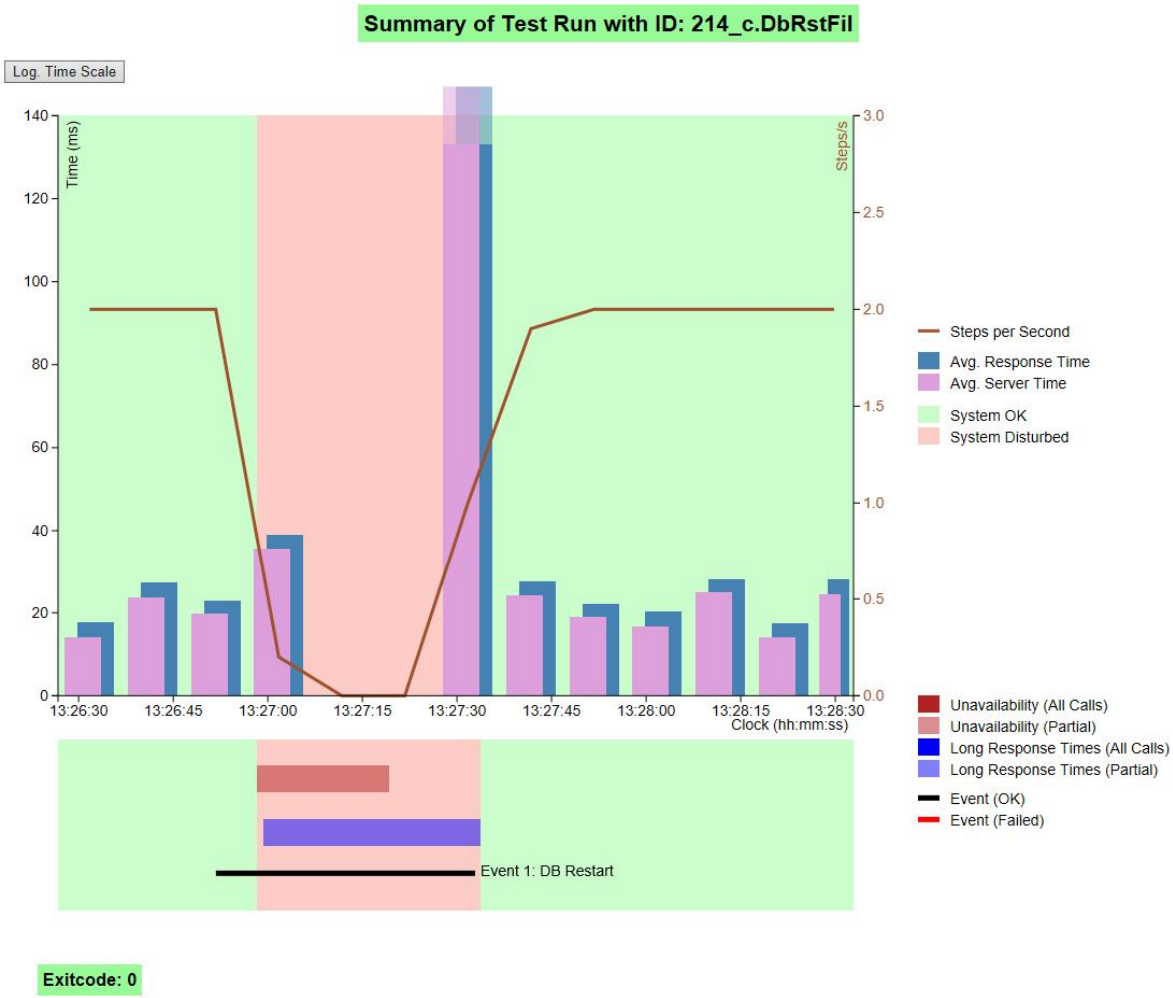
To display the HTML file, Java Script must be enabled in the browser, and the graphics library used by ***loadtest.html*** must be accessible. The graphics library was specified by the property ***graphicslibrary*** for the test run (see chapter 5.1.1.12). Its location is shown at the beginning of ***loadtest.html*** in the lines

<script src="*<full path of graphics library>*"></script>
<script src="file:*//<full path of graphics library>*"></script>

As of version 2.14, two script tags are used to support different browsers.

Problems during the generation of ***loadtest.html*** are displayed in the log file ***loadtest.log.***

As an example, the output of a test run during which a DB restart was done is shown below:



loadtest.html has the following structure:

- Header: Testrunid of test run
  - Green background: test run was ok
  - Red background: test run finished with errors
- Chart
  - Depicts the high load phase of the test run
  - Light green background: system ok
  - Light red background: system disturbed (periods of unavailability or long response times occurred)
  - Upper part
    - Steps per second
    - Average response and server time
    - Logarithmic time scale offered if response time exceeds the linear time scale
  - Lower part (optional)
    - Displayed if the system was disturbed for some time or if events were executed
    - Periods of unavailability
    - Periods of long response times
    - Events
- Additional information (in text form; buttons to expand and collapse information)
  - Exitcode

- Check against expected results
  - Displayed only if a check was done
- Events
  - Displayed only in the case of erroneous events
- Verification result
  - Displayed only in the case of verification errors
- Errors
  - Displayed only in the case of errors
- Properties used for the test run

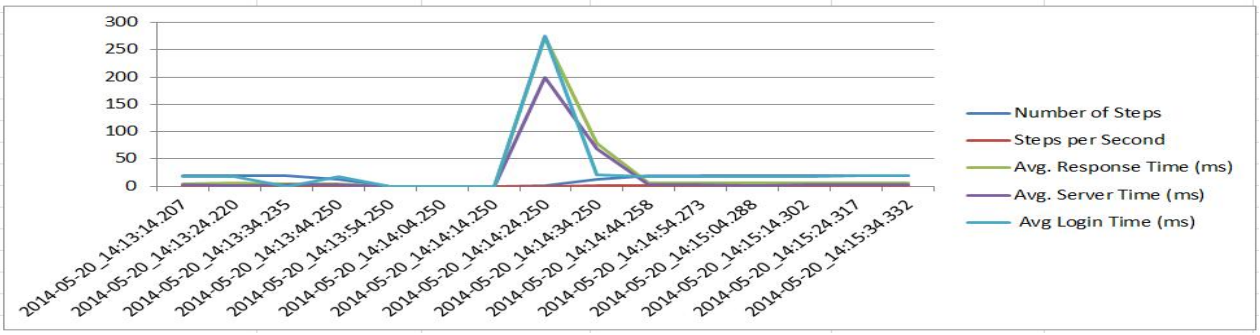
6.2.5 .csv File

The file ***loadtest.perf.csv*** contains performance and throughput information for the tacts in a csv format so that it can easily be shown by tools like Excel. The data is the same than the one that is shown for tacts in the performance and throughput section of the summary file (chapter 6.2.1).

The .csv file is offered temporarily only. Because a graphical representation of the data is available as of version 2.12 (see chapter 6.2.4), we plan to remove the .csv file in a future version.

In Excel, the file might look as follows (the chart was created manually):

Start of Tact	Number of Steps	Steps per Second	Avg. Response Time (ms)	Avg. Server Time (ms)	Avg Login Time (ms)
2014-05-20_14:13:14.207	20	2	5,3	2,2	17,6
2014-05-20_14:13:24.220	20	2	5,9	2	17,2
2014-05-20_14:13:34.235	20	2	5,3	2,1	0
2014-05-20_14:13:44.250	12	1,2	5,3	2,3	17
2014-05-20_14:13:54.250	0	0	0	0	0
2014-05-20_14:14:04.250	0	0	0	0	0
2014-05-20_14:14:14.250	0	0	0	0	0
2014-05-20_14:14:24.250	1	0,1	274	200	275,6
2014-05-20_14:14:34.250	13	1,3	79,1	68,8	20,5
2014-05-20_14:14:44.258	19	1,9	5,9	2,1	17,5
2014-05-20_14:14:54.273	20	2	5,8	2,3	18,5
2014-05-20_14:15:04.288	20	2	5,4	2	17,5
2014-05-20_14:15:14.302	20	2	5,6	2,2	17,8
2014-05-20_14:15:24.317	20	2	6,2	2,5	19
2014-05-20_14:15:34.332	20	2	6,2	2,4	19,5



6.3. Output Data of the Client Users

Each client user produces four output files named ***userxxxxx.yyy*** as shown in chapter 6.1. They allow a detailed analysis of the behavior of each simulated user.

6.3.1 Summary File

The file ***userxxxxx.summary*** (for example, user00001.summary) summarizes the test run of one single client user. It contains the following information:

- Start time of the test run
- Start and end time of the simulation of this client user
- Information about periods of erroneous calls indicating the unavailability of the system or system components
- Information about long response times
- Error overview of the calls of this client
- Data about performance and throughput of this client for the whole high load phase (only successful calls are taken into account)

Example of ***userxxxxx.summary***:

```
2017-12-21 13:25:35.439 Start time of test
2017-12-21 13:25:42.734 Client 7 started
2017-12-21 13:28:48.175 Client 7 finished successfully


D I S T U R B A N C E, indicated by E R R O N E O U S  C A L L S  (U N A V A I L B I L I T Y)
-------------------------------------------------------------------------------------------

No severe errors:                  2017-12-21 13:25:42.735 - 2017-12-21 13:26:53.260
Severe errors in all calls:        2017-12-21 13:26:58.261 - 2017-12-21 13:27:18.296  (20,035s)
No severe errors:                  2017-12-21 13:27:23.571 - 2017-12-21 13:28:48.164


D I S T U R B A N C E, indicated by L O N G  R E S P O N S E  T I M E S
-----------------------------------------------------------------------

Threshold for long response times = 10 seconds

No response:                  2017-12-21 13:27:23.571 - 2017-12-21 13:27:33.807  (10,236s)


E R R O R  O V E R V I E W
--------------------------

Number of erroneous calls:    1

Errors during execution of steps
  Number of application errors (APP_NOTAVAIL_ERR): 1


P E R F O R M A N C E and T H R O U G H P U T
---------------------------------------------

Time range:  2017-12-21 13:26:26.735 - 2017-12-21 13:28:32.888
Number of steps:    18
Number of logins:   4
Steps per second:   0,1
Avg. response time in ms:          592,9
Avg. elapsed time(Server) in ms:   23,5
Avg. response time of login in ms: 14,2
```

6.3.2 Response File

The file ***userxxxxx.res*** contains information about each single call to the server.

Example of ***userxxxxx.res***:

```
2017-12-21 13:25:35.439 Start time of test
2017-12-21 13:25:42.734 Client 7 started

Format of lines for EXEC calls:
Time stamp | OK/ERR | call | loop number | step number | elapsed time (server) | elapsed time (all)

Format of lines for LOGIN/LOGOFF calls:
Time stamp | OK/ERR | call | elapsed time (all)

2017-12-21 13:25:42.735 OK   LOGIN  0,015
2017-12-21 13:25:47.751 OK         EXEC   1 1 0,027 0,030
2017-12-21 13:25:52.782 OK         EXEC   1 2 0,038 0,040
2017-12-21 13:25:57.822 OK         EXEC   1 3 0,028 0,031
2017-12-21 13:26:02.853 OK         EXEC   1 4 0,024 0,026
2017-12-21 13:26:07.880 OK         EXEC   1 5 0,012 0,014
2017-12-21 13:26:07.897 OK   LOGOFF 0,011
2017-12-21 13:26:07.910 OK   LOGIN  0,014
2017-12-21 13:26:12.924 OK         EXEC   2 1 0,014 0,021
2017-12-21 13:26:17.945 OK         EXEC   2 2 0,038 0,041
2017-12-21 13:26:22.987 OK         EXEC   2 3 0,089 0,091
High load phase started
2017-12-21 13:26:28.079 OK         EXEC   2 4 0,024 0,027
2017-12-21 13:26:33.106 OK         EXEC   2 5 0,011 0,013
2017-12-21 13:26:33.120 OK   LOGOFF 0,010
2017-12-21 13:26:33.132 OK   LOGIN  0,014
2017-12-21 13:26:38.146 OK         EXEC   3 1 0,014 0,018
2017-12-21 13:26:43.164 OK         EXEC   3 2 0,037 0,040
2017-12-21 13:26:48.204 OK         EXEC   3 3 0,028 0,030
2017-12-21 13:26:53.234 OK         EXEC   3 4 0,024 0,026
2017-12-21 13:26:58.261 APP_NOTAVAIL_ERR EXEC   3 5 -1,000 20,035
2017-12-21 13:27:23.571 OK         EXEC   3 5 0,042 10,236
2017-12-21 13:27:33.808 OK   LOGOFF 0,010
2017-12-21 13:27:33.820 OK   LOGIN  0,016
2017-12-21 13:27:37.709 OK         EXEC   4 1 0,013 0,016
2017-12-21 13:27:42.726 OK         EXEC   4 2 0,037 0,040
2017-12-21 13:27:47.767 OK         EXEC   4 3 0,029 0,032
2017-12-21 13:27:52.799 OK         EXEC   4 4 0,023 0,025
2017-12-21 13:27:57.824 OK         EXEC   4 5 0,011 0,014
2017-12-21 13:27:57.840 OK   LOGOFF 0,010

.....
2017-12-21 13:28:23.016 OK   LOGIN  0,014
2017-12-21 13:28:28.030 OK         EXEC   6 1 0,016 0,020
High load phase ended
2017-12-21 13:28:33.050 OK         EXEC   6 2 0,039 0,042
2017-12-21 13:28:38.092 OK         EXEC   6 3 0,029 0,032
2017-12-21 13:28:43.124 OK         EXEC   6 4 0,024 0,027
2017-12-21 13:28:48.151 OK         EXEC   6 5 0,009 0,013
2017-12-21 13:28:48.165 OK   LOGOFF 0,010
2017-12-21 13:28:48.175 Client 7 finished successfully
```

In the middle of the response file above, you see the erroneous call during the restart phase of the database. Then the step in which the error occurred was repeated and waited until the reconnect was possible. It finished successfully, but the call was added to the periods of long response times. Afterwards no problems were discovered.

6.3.3 Error File (Overview)

In file ***userxxxxx.err*** you find an extract of all erroneous calls shown in file ***userxxxxx.res.*** If no error occurred, this file is empty.

Example of ***userxxxxx.err***:

```
2017-12-21 13:26:58.261 APP_NOTAVAIL_ERR EXEC   3 5 -1,000 20,035
-----------------------------------------------------------------------------------------------
```

A more complicated example is shown below. Here a communication problem causes an inconsistency later in the test. You see that a sequence of consecutive erroneous calls is confined by a dashed line (----------).

```
2014-10-09 09:07:30.333 COMM_ERR   EXEC   4 4 -1,000 0,002
2014-10-09 09:07:34.828 ERR  LOGIN  1,073
2014-10-09 09:07:41.272 ERR  LOGIN  1,210
2014-10-09 09:07:49.756 ERR  LOGIN  0,004
2014-10-09 09:07:57.205 ERR  LOGIN  0,004
-----------------------------------------------------------------------------------------------
2014-10-09 09:08:11.412 APP_INCONSISTENT_ERR EXEC   4 5 0,042 0,045
-----------------------------------------------------------------------------------------------
```

6.3.4 Error File (Detail)

The file ***userxxxxx.elg*** shows the stack traces of the erroneous calls listed in ***userxxxxx.err***.

For each error, the time stamp and error code is shown in both files. This information can be used to find the right stack trace for an error shown in ***userxxxxx.err***.

Example of ***userxxxxx.elg***:

```
2017-12-21 13:26:58.261 APP_NOTAVAIL_ERR    ******* Start of error log entry *******
00024No connect to database, session terminated??????????????????????????
com.sap.conn.jco.JCoException: (104) JCO_ERROR_SYSTEM_FAILURE: 00024No connect to database, session terminated??????????????????????????
  at com.sap.conn.jco.rt.MiddlewareJavaRfc.generateJCoException(MiddlewareJavaRfc.java:670)
  at com.sap.conn.jco.rt.MiddlewareJavaRfc$JavaRfcClient.execute(MiddlewareJavaRfc.java:1881)
  at com.sap.conn.jco.rt.ClientConnection.execute(ClientConnection.java:1120)
  at com.sap.conn.jco.rt.ClientConnection.execute(ClientConnection.java:953)
  at com.sap.conn.jco.rt.RfcDestination.execute(RfcDestination.java:1465)
  at com.sap.conn.jco.rt.RfcDestination.execute(RfcDestination.java:1435)
  at com.sap.conn.jco.rt.AbapFunction.execute(AbapFunction.java:300)
  at com.sap.test.haload.CallTypeActionsRFCStateful.execStep(CallTypeActionsRFCStateful.java:363)
  at com.sap.test.haload.CallTypeActions.execSession(CallTypeActions.java:213)
  at com.sap.test.haload.ClientThread.run(ClientThread.java:49)
Caused by: RfcException: [SID|xxxxxxxx]
    message: 00024No connect to database, session terminated??????????????????????????
    Return code: RFC_SYS_EXCEPTION(3)
    error group: 104
    key: RFC_ERROR_SYSTEM_FAILURE
Exception raised by SID|xxxxxxxx

  at com.sap.conn.rfc.engine.RfcIoOpenCntl.RfcReceive(RfcIoOpenCntl.java:2086)
  at com.sap.conn.jco.rt.MiddlewareJavaRfc$JavaRfcClient.execute(MiddlewareJavaRfc.java:1829)
  ... 8 more
Caused by: com.sap.conn.rfc.exceptions.RfcGetException: 00024No connect to database, session terminated??????????????????????????
  at com.sap.conn.rfc.engine.RfcGet.rfcget_gethead(RfcGet.java:459)
  at com.sap.conn.rfc.engine.RfcGet.ab_rfcget(RfcGet.java:35)
  at com.sap.conn.rfc.engine.RfcRcv.ab_rfcreceive(RfcRcv.java:33)
  at com.sap.conn.rfc.engine.RfcIoOpenCntl.RfcReceive(RfcIoOpenCntl.java:1981)
  ... 9 more
Caused by:
>RfcIoException:
    message: 00024No connect to database, session terminated??????????????????????????
    Return code: RFCIO_ERROR_DEALLOCATED_REMOTE_ERROR(33)<


  at com.sap.conn.rfc.engine.RfcIoOpenCntl.ab_rfcread(RfcIoOpenCntl.java:706)
  at com.sap.conn.rfc.engine.RfcGet.rfcget_gethead(RfcGet.java:455)
  ... 12 more
-  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -  -
-----------------------------------------------------------------------------------------------
```

6.4. Output Data of the HA Events

For each HA event two output files named ***eventxxx.yyy*** are written to subdirectory ***event*** as shown in chapter 6.1.

6.4.1 Log File Written by the HA Test Tool

The HA Test Tool starts events specified by the property ***eventcall*** (see chapter 5.1.1) and waits for the end of the events. All properties of an event, its start and its end are logged by the tool to file ***eventxxx.log*** (for example, event001.log for event 1).

Example of such a file:

```
Properties of HA Event 1:
  eventname:           DB Restart
  eventstarthighload:  25
  eventdefinitionfile: C:\HATool\config\event.db.restart.txt
  eventcall:           C:\HATool\event\db.restart.bat

2017-12-21 13:26:51.772 Event 1 started

2017-12-21 13:27:32.886 Event 1 finished with returncode 0
```

6.4.2 Output File Written by the HA Event

The output of an event (stdout and stderr) is redirected to file ***eventxxx.out.*** The event script or program is not part of the HA Test Tool package. If you provide such a script or program you should take care of meaningful output so that you can analyze the event afterwards.

In the example of chapter 6.4.1, the script db.restart.bat produces the following output:

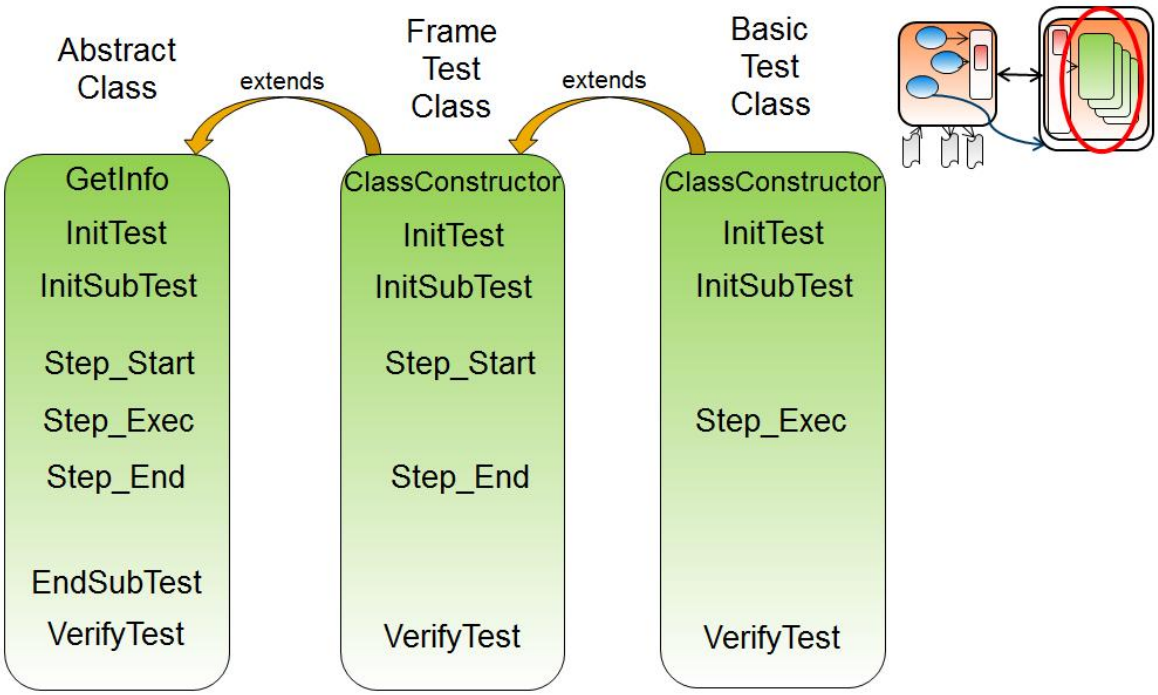
```
C:\HATool>C:\usr\sap\hostctrl\saphostctrl -user sidadm xxxxxx -host xxxxxxxx -function StopDatabase -dbname SID ...
Webmethod returned successfully
Operation ID: 00160A11CAC403D6A8B1C061AF510043

----- Log messages ----
Info: saphostcontrol: Executing StopDatabase
Info: saphostcontrol: StopDatabase successfully executed

C:\HATool>rem wait for 20 seconds

C:\HATool>ping -n 21 localhost  1>nul

C:\HATool>C:\usr\sap\hostctrl\saphostctrl -user sidadm xxxxxx -host xxxxxxxx -function StartDatabase -dbname SID ...
Webmethod returned successfully
Operation ID: 00160A11CAC403D6A8B1C275AD7CC044

----- Log messages ----
Info: saphostcontrol: Executing StartDatabase
Info: saphostcontrol: StartDatabase successfully executed
```

7. Test Classes

If you want to get a deeper understanding of how the HA Test Tool is working on server side and of what the tests included in the package do, you should read the following chapter. It describes the principles of the test class hierarchy and provides details about the test classes.

7.1. Test Class Hierarchy

The following figure describes the hierarchy of the test classes:



There are three levels of test classes:

1. Class that defines the test interface
   - Implemented as an abstract class (named ***CL_HA_TEST*** in case of an ABAP server)
   - Contains the complete interface of the test classes
   - Already implements some functions thatare common to all tests (mainly in the area of test administration). These functions are not explained in detail here because understanding them is not required to understand the tests.
2. Frame Test Class
   - Extends the abstract test class
   - Contains a test implementation, but the main intention is to provide a test frame for a group of additional tests that run within this frame.
   - The current HA Test Tool contains the following frame test classes:
     i. ***CL_HA_TESTFRAME_ENQ*** (provides an enqueue test frame)
     ii. ***CL_HA_TESTFRAME_DB*** (provides a database test frame)
     iii. ***CL_HA_TESTFRAME_FILE*** (provides a file system test frame)
3. Basic Test Class
   - Extends a frame test class
   - Implements a test that runs under the umbrella of a frame test class
   - Easier to implement than a frame test class because the frame test class takes care of some general test topics (like error or transaction handling).
   - The current HA Test Tool contains one such test class. ***CL_HA_DB_LOAD*** uses ***CL_HA_TESTFRAME_DB*** and is used for database load tests.

Note that the picture above is a simplification. In reality the methods InitTest, InitSubTest and VerifyTest are split into three methods (…_START, …_EXEC, …_END; like STEP_...). Only the …_EXEC method has to be implemented by the derived test classes.

7.2. Tests Included in the HA Test Tool Package

7.2.1 Enqueue Frame Test Class

CL_HA_TESTFRAME_ENQ is a test class that is used to verify the enqueue functionality and additionally accesses a database table that corresponds to the enqueue object.

This test class is a so called 'frame test class'. This means that the class can be used to build further test classes that run inside the frame of their superclass.

According to the general call sequence described in chapter 2.2, ***CL_HA_TESTFRAME_ENQ*** does the following:

- ExecStep 1
  - Create specific enqueue(s) for current loop containing the steps 1–n (lock object: ***EHALOADFRAME***). By default one enqueue is created, but with property ***subtestparameters=EnqueuesPerLoop:nnn*** you can increase that number up to 10000. Keep in mind that a big number puts lot of burden on the enqueue server and might require a bigger size of the lock table (size set by (server parameter enque/table_size).
  - Insert row in database table ***HALOAD_ENQ_TAB***
    - Primary key specifies current loop
    - Column ***STEP_NUM*** set to 1 (current step number)
    - Column ***ENQ_NUM*** set to number of enqueues per loop
- ExecStep 2 – (n-1)
  - Update row in database table ***HALOAD_ENQ_TAB***
    - Set column ***STEP_NUM*** to current step number
- ExecStep n (last step of the loop)
  - Dequeue locks that were set in step 1
    - If a lock was not found, throw an inconsistency error.
  - Delete row that was inserted in step 1 from table ***HALOAD_ENQ_TAB***
    - If a row was not found, throw an inconsistency error.
- Step repetition because of previous error (COMM_ERR or APP_NOTAVAIL_ERR)
  - Because of the system design, the enqueues of the current loop have been removed. Therefore, they are created again as described in 'ExecStep 1' above to avoid verification errors in the summary of the test run. This behavior was introduced with version 2.11. Verification errors should be restricted to cases in which the system seems to behave as expected, but produces inconsistencies.
  - As described in chapter 7.3.1, the failover of the enqueue server should not cause errors. Therefore, the occurrence of the error categories COMM_ERR or APP_NOTAVAIL_ERR would still indicate that the scenario has failed.

At the end of the test run, the verification function is called. It returns ok if no lock or no table entry set by this test run is found.

7.2.2 Database Frame Test Class

CL_HA_TESTFRAME_DB is a test class that provides a frame for database load tests. It does not generate a reasonable load by itself, but can be used as a superclass for classes that put load on the database. ***CL_HA_DB_LOAD*** is such a class (see chapter 7.2.3).

The main purpose of ***CL_HA_TESTFRAME_DB*** is to initially fill some database tables with data and to run 'update statistics' on these tables. Subclasses can copy these data and use it for their load test. They are not allowed to modify the data of ***CL_HA_TESTFRAME_DB*** directly. The subclasses

have to be aware that ***INIT_TEST_EXEC*** might return ***ret_code=SC_ERR_INFORMATION*** if the 'update statistics' call by function ***RSDU_ANALYZE_TABLE*** is not supported by the database. Processing should proceed, but the information will be reported to the client.

The following database tables are filled by ***CL_HA_TESTFRAME_DB***:

- ***HALOAD_DB_MAT***: material table; 1000 rows are filled
- ***HALOAD_DB_CUST***: customer table; 1000 rows are filled
- ***HALOAD_DB_ORDER***: order table; 1000 rows are filled
- ***HALOAD_DB_ORDITM***: order item table; 10000 rows are filled

To have disjunct data for ***CL_HA_TESTFRAME_DB*** and its subclasses, the following table scheme was introduced:

- The first column of each table is named ***INT_RUNID***.
- ***CL_HA_TESTFRAME_DB*** sets int_runid to 0.
- A subclass, like ***CL_HA_DB_LOAD*** has to use the internal ID of the test run. This value is generated by the abstract class of the test environment. It is unique and greater than 0.

There is no need to use ***CL_HA_TESTFRAME_DB*** directly for a test run. Its subclasses have to call the method ***INIT_TEST_EXEC*** of their superclass during initialization, so that the tables above are filled. If this was already done by a previous run, it will not be done again.

There is only one reason to use ***CL_HA_TESTFRAME_DB*** directly with the HA Test Tool:

If you want to empty the tables used for the database load tests, you should set the property ***specialaction=ResetTestData***, omit property ***testparameters*** (see chapter 5.1.1) and make a run with each of the subclasses of ***CL_HA_TESTFRAME_DB*** (in the HA Test Tool package, this is just ***CL_HA_DB_LOAD***) and finally with ***CL_HA_TESTFRAME_DB***. Note that resetting the data of ***CL_HA_DB_LOAD*** will not delete rows created by ***CL_HA_TESTFRAME_DB***.

7.2.3 Database Load Test Class

In order to perform high availability tests under database load, the HA Test Tool package contains the test class ***CL_HA_DB_LOAD.*** With this class, you can put configurable load on the database. Additionally, it performs data consistency checks during and at the end of the run.

CL_HA_DB_LOAD works as follows:

- Used tables
  - The tables that are listed in chapter 7.2.2 are used: ***HALOAD_DB_MAT, HALOAD_DB_CUST, HALOAD_DB_ORDER*** and ***HALOAD_DB_ORDITM***
  - The results of the checks are written to ***HALOAD_DB_CHECK.*** This table has, like the other ones above, the internal run ID as the first column so that all data is test-run specific.
  - The tables ***HALOAD_CALLCHECK*** and ***HALOAD_SUBPARAMS*** that are provided by the abstract class of the test framework (see chapter 7.1) are used to prevent calls from double execution and to store parameters for the load test.
- Configuration of the load
  - As for any other test you can use the property ***clientnum*** (see chapter 5.1.1) to specify how many client users are simulated.
  - You can use the property ***subtestparameters=ItemsPerOrder:nnn*** (nnn in 1 .. 10000; see chapter 5.1.1) to specify the number of order items per order. This value is approximately proportional to the load that is generated per step.
- Initialization of data
  - There is no special initialization for the complete test run (method ***INIT_TEST_EXEC)*** beside calling the corresponding method of the superclass (***CL_HA_TESTFRAME_DB***) and resetting data of previous test runs.
  - During the initialization for the subtest run (method ***INIT_SUBTEST_EXEC***), the data of the tables ***HALOAD_DB_MAT*** and ***HALOAD_DB_CUST*** provided by the frame test class ***CL_HA_TESTFRAME_DB*** (int_runid = 0) is copied. The column int_runid is set to the internal ID of the subtest run. This ID is provided by the test framework. The other ***HALOAD_DB_xxx*** tables are not changed.

- Execution of the test
  - The test consists of five different steps that are called by method ***STEP_EXEC***:
    - INSERT step
    - UPDATE step
    - DELETE step
    - ROLLBACK step
    - CHECK step
  - According to the general call sequence described in chapter 2.2, ***CL_HA_DB_LOAD*** does the following: Each step is executed by a separate call from the client and processed in one transaction. In case of an error that indicates the unavailability of the system or the database, the client repeats the step until it is processed successfully.
  - The property ***stepnum*** (see chapter 5.1.1) can have a value from 2 to 5. This means the following for the execution of the steps:
    - stepnum = 2: insert + check
    - stepnum = 3: insert + update + check
    - stepnum = 4: insert + update + delete + check
    - stepnum = 5: insert + update + delete + rollback + check
  - During one sequence of steps an order with order items is created using data of the material and customer tables. Order and order items are updated afterwards. Then half of the order items are deleted. A new order is inserted, but the transaction is rolled back afterwards. Data consistency checks are done during each step and, in addition, in the last step of the sequence. The number of order items per order is 10 per default, but can be changed for the subtest run by the property ***subtestparameters=ItemsPerOrder:nnn*** (nnn in 1 .. 10000; see chapter 5.1.1). The material and customer tables are not updated, but only used to provide data for certain columns of the order and order item tables and to do checks during the run. In detail, during the steps, the following is done:
    - ***INSERT step***
      - One order and its order items are created. They are used in the following steps of the call sequence. In order to make them unique, the first three columns (INT_RUNID, USER_NUM and ORDER_NUM) are set to the internal ID of the subtest run, the number of the calling client user and the loop number (see picture in chapter 2.2). The next column of the order item table is ORDER_POS, whose value is in the range between 1 and the maximum number of order items per order.
      - For the order item table (***HALOAD_DB_ORDITM)*** the columns MAT_NUM, PRICE and DELIVERY_TIME of the material table (***HALOAD_DB_MAT***) are used. If ItemsPerOrder is greater than 20, all materials for the current internal ID are read with an array select, otherwise a single select is done for each used material.
      - For the order (table ***HALOAD_DB_ORDER)*** the column CUST_NUM of table ***HALOAD_DB_CUST*** is used. DELIVERY_DATE is calculated using the maximum delivery time of the order items and TOTAL_PRICE is the sum of all item prices.
      - For the order a single insert is used, for the order items an array insert.
      - As well as for the other steps, consistency checks are done, and the results are inserted into table ***HALOAD_DB_CHECK*** (for example, cross checks for price and delivery date).
    - ***UPDATE step***
      - The order and its order items are selected.
      - For each order item, the amount of used materials is doubled and therefore also the item price and the total price of the order.
      - Additionally, the action ‘Price doubled’ is written to a varchar column (REMARK_HEADER) of the order and order item records. The

column was previously empty. A CLOB column (REMARK) of the order record is now also filled with a fixed part, describing the action and a random string. Up to 8 KB are filled into this column.
    - ***HALOAD_DB_ORDER*** and ***HALOAD_DB_ORDITM*** are updated (the latter with an array update). Check results are inserted into ***HALOAD_DB_CHECK.***
  - ***DELETE step***
    - The order and its order items are selected.
    - All order items with an even value of ORDER_POS are deleted (by an array delete from ***HALOAD_DB_ORDITM***).
    - The total price and the delivery date of the order have to be adapted according to the remaining set of order items.
    - The new action 'Half of order items deleted' is added to the columns REMARK and REMARK_HEADER of the order. Additionally, the random part of REMARK is changed.
    - The order record is updated in ***HALOAD_DB_ORDER*** and the check results are written to ***HALOAD_DB_CHECK.***
  - ***ROLLBACK step***
    - Duplicate key check: The insert step above is repeated with the same values. If no duplicate key error occurs, the message that the test has failed will be written to ***HALOAD_DB_CHECK*** and the step will be committed and finished.
    - Rollback check: The insert step above is repeated with a bigger value of column ORDER_NUM. It is expected that the insert will be successful. Afterwards the transaction is rollbacked.
  - ***CHECK step***
    - Several checks are done and the results are inserted into ***HALOAD_DB_CHECK.*** There are checks like these:
      - Does the order exist?
      - Is the number of order items ok?
      - Are the values of TOTAL_PRICE and DELIVERY_DATE in the order record ok?
      - Is the length of the CLOB column REMARK ok?
- Verification at the end of the test
  - The final verification is done with method ***VERIFY_TEST_EXEC***.
  - It is checked that the number of rows in the tables ***HALOAD_DB_MAT, HALOAD_DB_CUST, HALOAD_DB_ORDER,*** and ***HALOAD_DB_ORDITM*** is ok.
  - It will be indicated, if an error occurred in any of the checks made in the steps above.

7.2.4 File System Frame Test Class

CL_HA_TESTFRAME_FILE is a test class that provides a test for the accessibility of the shared file system used by the SAP system.

Although this test class is implemented as a 'frame test class', it is not really intended to be extended by basic test classes. ***CL_HA_TESTFRAME_FILE*** opens and closes a specific file in the shared file system. It does not allow to create, read or write arbitrary files because of security reasons and because it is not really necessary for the test of the shared file system.

CL_HA_TESTFRAME_FILE works as follows:

- Used resources
  - The file ***DEFAULT.PFL*** is accessed.
  - This file is located in the profile directory of the SAP system. The directory, which is specified by the system parameter ***$(DIR_PROFILE)***, is typically set to ***/usr/sap/<SID>/SYS/profile*** and located in a shared file system.
- Configuration of the load

    - No test-specific configuration is possible. Only parameters as for any other test, like the number of users or steps, can be set.
- Initialization of data
    - No special initialization is done.
    - The test just checks that the file ***DEFAULT.PFL*** exists in the profile directory. If this is not the case, the test will not be started.
- Execution of the test
    - Each step of the test does the same:
        - ***STEP_START*** opens the file ***DEFAULT.PFL***.
        - ***STEP_END*** closes the file after two seconds.
        - No data is read or written.
        - If the steps fail, the error ***APP_NOTAVAIL_ERR*** is returned because it is assumed that the file system is not available.
- Verification at the end of the test
    - Because no data is written, there is no inconsistency check at the end of the test. The verification always delivers 'OK'. Whether the test behaves as expected has to be determined by the outcome of the execution of the steps.

7.3. Expected Behavior of Test Classes

The test classes ***CL_HA_TESTFRAME_ENQ*** (see chapter 7.2.1) and ***CL_HA_DB_LOAD*** (see chapter 7.2.3) can be used for load tests. Before making tests with HA events, you should check that the system is able to run the intended load without errors and performance problems. This means that the exit code of the test run has to be 0 (no verification errors, no errors at all, and no long response times).

When using HA events, the expected result depends on the test and on the event. But in all cases, for a successful run no fatal error, no verification error and no HA event problem must occur. This would be indicated by the exit code of the test run (see chapter 6.2.2) and by the corresponding sections of file ***loadtest.summary*** (see chapter 6.2.1).

7.3.1 Enqueue Frame Test Class

The enqueue test class ***CL_HA_TESTFRAME_ENQ*** is normally used to test HA scenarios with the SCS (SAP Central Services) instance. In case of a failover of the enqueue server, the following behavior is expected from the server:

- Calls to the enqueue server are blocked until it becomes operational again. If this time is longer than the long response time threshold (specified with property ***noresponsethreshold***; see chapter 5.1.1), the time during which calls are blocked is indicated as period of long response times. However, the calls should not return errors.
- If you set property ***noresponsethreshold*** to the time that is acceptable for you for failover, the system behaves as expected if no long response times occur. Since the test run must not show any errors, the exit code of the run has to be 0.
- If you set property ***noresponsethreshold*** to a low value to see how long the system is disturbed by the failover, you will get exit code 1 (long response times; see chapter 6.2.2). In file ***loadtest.summary*** (see chapter 6.2.1), the section ***Availability and Responsiveness*** shows how long the system was disturbed, and the reason is described in section ***Disturbance, indicated by Long Response Times***.

7.3.2 Database Load Test Class

The database load test class ***CL_HA_DB_LOAD*** is normally used to test HA scenarios with the database. In case of a failover of the database server, the following behavior is expected:

- Calls that have opened a transaction on the database are aborted and return an error. This kind of errors is indicated by the test tool as a server disturbance because a server component (the database) is not available.

- When the application server realizes that the database is not operational, calls that arrive later are blocked until a reconnect to the database is possible again. If the duration is longer than the long response time threshold (property ***noresponsethreshold***; see chapter 5.1.1), it is indicated as a period of long response times. However, these calls do not return any errors.
- If you set property ***noresponsethreshold*** to the time that is acceptable for you, the system behaves as expected if only errors occur that indicate that the system or sytem components are not available (error class ***APP_NOTAVAIL_ERR*** in file ***loadtest.summary***; see chapter 6.2.2). The exit code is 6 (erroneous calls, unavailability; see chapter 6.2.2). In file ***loadtest.summary*** the sections ***Availability and Responsiveness*** and ***Disturbance, indicated by Erroneous Calls (Unavailability)*** show how long the period with erroneous calls lasts. However, the whole failover time is not indicated, because the period with blocked calls is not measured. You just know that it is below your threshold.
- If you set property ***noresponsethreshold*** to a low value to see how long the system is disturbed by the failover, you will get exit code 7 (long response times, erroneous calls, unavailability; see chapter 6.2.2). In file ***loadtest.summary*** (see chapter 6.2.1) section ***Availability and Responsiveness*** shows how long the system was disturbed, and the reason is described in sections ***Disturbance, indicated by Erroneous Calls (Unavailability)*** and ***Disturbance, indicated by Long Response Times***. In the section ***Error Overview***, errors of category ***APP_NOTAVAIL_ERR*** are shown.
- In rare cases you might get errors of the categories ***COMM_ERR*** or ***ERR***. In the files of the client users (see chapter 6.3), you can check whether the missing connection to the database is the reason. In this case, the errors would be ok.
- For other errors, to have a deeper look at the cause of erroneous calls, you should also check the log files of the client users.

7.3.3 File System Frame Test Class

The file system test class ***CL_HA_TESTFRAME_FILE*** is normally used to test HA scenarios with the shared file system. In case of an unavailability of the shared file system, the following behavior is expected:

- When a call tries to open the file ***DEFAULT.PFL,*** an error of the category ***APP_NOTAVAIL_ERR*** is returned.
- In file ***loadtest.summary*** (see chapter 6.2.1), the section ***Availability and Responsiveness*** shows how long the system was disturbed. The reason is described in the section ***Disturbance, indicated by Erroneous Calls (Unavailability)***. In the section ***Error Overview***, errors of category ***APP_NOTAVAIL_ERR*** are shown.
- For other errors, to have a deeper look at the cause of erroneous calls, you should also check the log files of the client users.

8. Deinstallation

If you want to remove the HA Test Tool from your server, you need to perform the following actions:

- HA Test Tool user
  - If you have created a user solely for the usage of the test tool (recommended), remove it (transaction ***SU01***).
  - If you need the user for other purposes, detach at least the role ***SAP_BC_HALOAD*** from this user (transaction ***SU01***).
- Role
  - The role ***SAP_BC_HALOAD*** is necessary to execute tests on the server. Remove it using transaction ***PFCG***.
- Service
  - The service ***hatool*** is necessary to process HTTP requests.
  - In transaction ***SICF***, enter ***hatool*** in the field ***Service Name*** of the filter for calling the ICF hierarchy and execute the query.
  - Right-click on service ***hatool*** and delete the service.
- Programs and data
  - All programs and data of the HA Test Tool belong to package ***SDB_HA***. The contents of the package can be deleted by importing the transport ***BENK000001***. Do the following (on the server, you have to be logged on as user ***<sid>adm***):
    - You find the transport in the directory to which you have extracted the installation package (see chapter 4).
    - Copy the file ***K000001.BEN*** from directory ***…/HATool/server/uninstall*** to ***/usr/sap/trans/cofiles*** and ***R000001.BEN*** to ***/usr/sap/trans/data*** on the server.
    - Import them with the transport management system of the server (you can use transaction ***STMS_IMPORT***).
    - If transport ***BENK000001*** is not shown in the import queue, you can add it by choosing ***Extras → Other requests → Add.***
    - Import the request. Make sure that the the field *Target Client* contains a valid client of the target system.
    - If the request has been imported successfully, you see in transaction ***SE80*** that the package ***SDB_HA*** is empty.

9. Special Aspects

9.1. Security Considerations

9.1.1 Client-Server Communication

The client-server communication can optionally be secured via SNC (Secure Network Communication) or HTTPS.

9.1.1.1 SNC (Secure Network Communication)

With SNC, you can

- Encrypt the communication.
- Authenticate via the SNC identity. So, you avoid the authentification via user and password.

The SNC documentation and an SNC library are not part of the HA Test Tool package. You can use any library that provides the GSS-API V2 standard interface and that fulfills the SAP requirements for SNC libraries. To set up SNC on client and server side, you should use SAP's security documentation concerning SNC and the documentation of the SNC library that you want to use. For the usage of this configuration by the HA Test Tool, see the section with the SNC properties in chapter 5.1.1.7.

9.1.1.2 HTTPS

The following authentication types are supported for HTTPS:

- Server authentication
  Trust store is used (if not set, JRE default is used).
  - With user/password
  - Without user/password
    User/password has to be set on server side in transaction SICF for service 'hatool'.
- Mutual authentication
  Trust store and keystore are used.

To set up HTTPS, you should use SAP's security documentation concerning SSL for the server side. On client side you can use the JDK program ***keytool*** to add the public certificate of the root CA to the trust store. For the keystore the client certifcate has to be in PKCS12 format. For the usage of this configuration by the HA Test Tool, see the section with the HTTPS properties in chapter 5.1.1.8.

9.1.2 Client-Side

With the HA Test Tool, you can put a massive load on the server or disturb the server by triggering HA events. We therefore recommend that you control the usage of the tool. To make this easier, a strict directory scheme is used for the tool. In the directory used for the installation (see chapter 4.2), the following subdirectories are created:

.../HATool	Base directory of the HA Test Tool at client side
|	
+ ...../***bin***	Contains the driver file ***haTestTool.jar*** and the html template
|	***haTestTool.html.template***
+ ...../***check***	Has to contain all files to check the results of test runs
|	(delivered with the package or written by yourself)
+ ...../***config***	Has to contain all property files for the tool
|	(delivered with the package or written by yourself)

```
+ …../event        Scripts or programs for HA events
 |                 (is delivered empty; content has to be provided by yourself)
+ …../output       Base directory for the output of the test tuns
 |
+ …../jcotrace     Used as location if JCo or RFC traces are switched on
```

These directories are mandatory for the corresponding files. Other directories are not used for them. Therefore you can restrict the usage of the tool by setting the access rights for this directory structure according to your needs.

9.1.3 Server-Side

On server side, each call to the tests is secured via an authorization object. The authorizations that are necessary to execute the tests are summarized in the role ***SAP_BC_HALOAD.*** Only users with this role can execute the tests. Other authorizations are not needed.

After your tests have finished, you can remove the HA Test Tool from the server by executing the steps described in chapter 8.

9.2. HA Events

In addition to generating load on a server, the HA Test Tool is able to trigger HA events during the high load phase. The HA events are scripts or programs that have to be located in the subdirectory ***event*** of the base directory of the tool. There are no preconfigured events delivered with the tool. You have to provide them yourself.

For the HA Test Tool the semantices of an HA event is transparent. The tool simply does the following (the properties below are described in detail in chapter 5.1.1):

- The call is done according to the property***eventcall.*** Here the name of the script or program and the parameters are specified. As of version 2.13, parameters can be variables. These variables can be test properties of the test run or user-defined properties whose values are specified by the property ***eventcallparameters*** (see chapter 5.1.1.10).
- The start time can be relative to the beginning of the high load phase (property ***eventstarthighload***) or relative to the end of the previous event (property ***eventstartprevious***).
- There are two output files for each event (located in the event subdirectory of the test run (***…HATool/output/<testrunid>/<clienthost>/<calltype>/<testclass>/event***; see also chapter 6.4):
  - ***eventxxx.log*** Log file of event xxx written by the test tool
  - ***eventxxx.out*** Output of event xxx written by the event

  In addition, the start, stop and exit code of each event is shown in file ***loadtest.log*** and an overview of all events is given in ***loadtest.summary***. Both files are located in ***…HATool/output/<testrunid>/<clienthost>/<calltype>/<testclass>*** (the base directory of the test run).

To make the event output easier to understand, a human readable name for the event has to be specified (property ***eventname***). This name is not relevant for the processing of the event. Since an event can be used in different tests, ***eventcall*** and ***eventname*** can be specified in a separate file (specified by property ***eventdefinitionfile***). The start time is not set here because it is specific to the test.

If you want to run the test a certain amount of time after the last event was processed, you can set the property ***durationafterevents*** to this value and specify the maximum duration of the whole run with the property ***duration***.

In the case of several events, event definition files have to be used. The properties ***eventstarthighload, eventstartprevious*** and ***eventdefinitionfile*** have to end with '***.nr***'. The events have to be numbered consecutively starting with nr=1.

You can specify isolated events that do not depend on other events or event chains. Here, an event is triggered (property ***eventstartprevious)*** relative to the end of the previous one if this event finished with exit code 0.

For a correct execution and analysis of the events, they should comply with the following rules:

- Exit code
  - 0: Event was executed successfully. The exit code belongs to the event only, not to the reaction of the server on it.
  - ≠ 0: Event was not executed successfully. In such a case,
    - The test run will indicate an 'HA Event problem'.
    - Successor events of an event chain will not be executed.
- Duration
  - Ideally, an event should return when the action triggered by the event is over. This has following advantages:
    - The exit code indicates that the event was processed successfully and not just the triggering.
    - In an event chain, it is easier to specify a meaningful value for ***eventstartprevious*** because during this period, the system has time to return to normal behavior.
    - The same is true when using property ***durationafterevents.***
    - In the analysis of the test run it is easier to correlate the event with the disturbance of the system when you know how long the event lasts.
  - It is intended to process the events during high load phase. This has to be taken into account when specifying the values for the properties ***duration, eventstarthighload*** and ***eventstartprevious***. If it is clear that not all events can be started during the high load phase because of these values, the test run will not be started. If an event has not finished at the end of the high load phase, the test will wait for some time before it stops. If the event is still not finished, this will be indicated in the test output, but the event itself is not disturbed and can continue to write its output to ***eventxxx.out***.

In most cases triggering one event per test run will be sufficient. However, to demonstrate the handling when using several events, the following example shows the usage with 4 events:

Overview

- Event 1 (isolated event)
- Event 2 and 3 (event chain)
- Event 4 (isolated event)

HA event-related properties

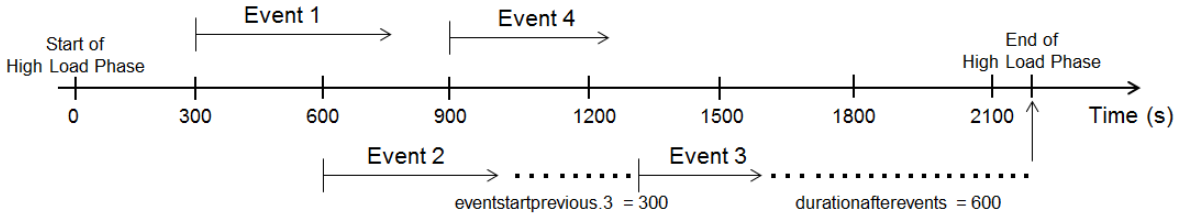
- duration = 3600
- durationafterevents = 600

- # Event 1
- eventstarthighload.1 = 300
- eventdefinitionfile.1= eventfile1 → eventname = Event 1
  eventcall = event1.sh param1 param2
- # Event 2
- eventstarthighload.2 = 600
- eventdefinitionfile.2= eventfile2 → eventname = Event 2
  eventcall = event2.sh a b c

- # Event 3
- eventstartprevious.3 = 300
- eventdefinitionfile.3= eventfile3 → eventname = Event 3
  eventcall = event3.sh 1 2 3 4
- # Event 4
- eventstarthighload.4 = 900
- eventdefinitionfile.4= eventfile4 → eventname = Event 4
  eventcall = event4.sh

Remark: The event definition files must be located in subdirectory ***config*** and the scripts must be in subdirectory ***event*** of the HA Test Tool base directory.

Timeline



Event 3 will be triggered only if Event 2 returns with exit code 0. Event 4 would be executed anyway because it is an isolated event that is not part of a chain.

9.3. Scalability Guidelines

This chapter gives you an overview of how to increase the load on the SAP Web AS ABAP with the HA Test Tool. The main focus is on aspects that are relevant for the HA Test Tool. For general performance and tuning guidelines for SAP Web AS ABAP, see the specific documentation.

For details about the properties mentioned below, see chapter 5.1.1.

9.3.1 General Recommendations

With the HA Test Tool, you can easily configure a load that might exceed the capacity of the server or the server configuration. For this reason, we give some general recommendations for setting up high availability (HA) tests under heavy load.

9.3.1.1 Step-by-Step Approach

We recommend to start with a small load without triggering HA events (for example, 10 users (property ***clientnum*** = 10), default value for ***thinktime*** and no explicit setting of ***subtestparameters***).

Only if this test runs without errors and with response times in the milliseconds range, you should start testing HA events or increase the load as described in chapter 9.3.2.

Here you should not make the load explode by making huge changes to the relevant properties. Increase the load in manageable steps and monitor the behavior of the system carefully.

Of course, no errors must occur because of the high load, and the average response times should not increase dramatically. Otherwise, you have to optimize the configuration of the system or to decrease the load.

9.3.1.2 Preparing HA Events

You should only trigger HA events in a system that runs smoothly. This means that a test run without HA events should not show any errors, the response times should be reasonably well, and there should be no resource bottlenecks.

An HA event can be triggered by the HA Test Tool calling a program or by a script provided by you. We recommend to test this script or program upfront without the tool to check whether it works as expected.

It might happen that after an HA event the calls of the users are issued almost at the same time. If you want to avoid this behavior, you can spread the calls by setting the properties ***errorthinktimelow*** and ***errorthinktimehigh***.

9.3.1.3 Monitoring of Resource Consumption

As usual for tests with high loads, the resource consumption of the system should be monitored carefully. Especially consider that the memory consumption increases with the number of users and can lead to swapping. The same might happen when you increase the number of work processes.

In particular for the HA Test Tool, you should take care of the following:

When using the DB load test (test class ***CL_HA_DB_LOAD***), a high volume of data can be inserted into the database. This is especially true if the property ***subtestparameters = ItemsPerOrder*** is set to a high value and the test runs for quite some time. When you set property ***resettestdata = 0***, data of previous test runs is not deleted and the database can become full by subsequent test runs. If you want to keep the data of specific old runs, check the reset possibilities of the property ***testparameters***.

The enqueue test (test class ***CL_HA_TESTFRAME_ENQ***) can generate a lot of enqueues, especially if the property ***subtestparameters = EnqueuesPerLoop*** is set to a high value. The maximum number of parallel existing enqueues is calculated as follows:

clientnum * EnqueuesPerLoop

If this number is too high, the enqueue table has to be adapted (parameter ***enque/table_size***).

9.3.1.4 Usage of Several Application Server Instances

For information about distributing the calls over several application server instances, see the corresponding documentation of SAP Web AS ABAP.

With transaction ***SM50***, you can monitor the work process usage of a single instance. With transaction ***SM51***, you can toggle between several instances.

calltype = RFCStateful

In the following section, you find a quick guide that describes how to use several instances with calltype = RFCStateful:

- Server side (use transaction ***SMLG***)
  - Create or use a logon group that includes the application server instances.
  - On the tab ‘Attributes’, perform the following actions:
    - Select ‘Ext. RFC-enabled’.
    - Choose the ‘Favorite Type’:
      - ‘W’ Weighted Round Robin
      - ‘R’ Round Robin (in case of equal performance of the instances)
  - Copy and save the changes.
- Client side
  - Configure the logon via the message server.
  - Set the property ***logongroup*** to the logon group on the server.

calltype = HTTPStateful

A good description of the HTTP load distribution can be found here:
http://help.sap.com/saphelp_nw70ehp1/helpdata/en/be/6041418e998739e10000000a1550b0/content.htm

If the description does not match your SAP NetWeaver release, refer to the corresponding documentation of your version.

9.3.2 Increasing the Load

If you want to test your HA environment not only with a basic load, you should know the possibilities to increase the load with the HA Test Tool. The different approaches are described in the following subsections. Of course, you can combine the possibilities to achieve a high load.

9.3.2.1 Load per Call

You can increase the load on the server side by increasing the load generated by single calls to the server. This does not mean that there is more communication traffic between client and server, but that the server has to spend more effort to process the calls. This can be done by using the property ***subtestparameters***. Its setting depends on the test class used for the run.

CL_HA_TESTFRAME_ENQ (Enqueue Test Class)
subtestparameters = EnqueuesPerLoop:nnn
The number of enqueues per loop can be specified. A value between 1 and 10.000 is allowed. If not set, one enqueue per loop is created. In the first step of the loop the enqueues are created, and in the last step they are released. To increase the content of the enqueue table and the load on the enqueue server, raise the value of ***EnqueuesPerLoop***. As described in chapter 9.3.1.3 (Monitoring of Resource Consumption), take care of the size of the enqueue table (parameter ***enque/table_size***). In the steps between the first and the last step of the loop the enqueue server is not used, but just a small load is put on the database. Therefore, a loop should consist of a few steps only (recommendation: set property ***stepnum*** to a value between 3 and 5).

CL_HA_DB_LOAD (Database Test Class)
subtestparameters = ItemsPerOrder:nnn
The number of order items per order can be specified. A value between 1 and 10.000 is allowed. If not set, 10 items per order are created. For the insert, the update, the delete and the rollback step, the load on the database and on the application server is approximately proportional to the value of ***ItemsPerOrder***. The check step (last step of the loop) is not significantly influenced by this value. As described in chapter 9.3.1.3 (Monitoring of Resource Consumption), make sure that the database does not become full. Unless you do not want to exclude certain modification steps, the test should be executed with ***stepnum*** 5 so that all different modification operations are executed. Then four out of five steps are heavily influenced by ***ItemsPerOrder***.

9.3.2.2 Call Frequency per User

Another possibility to increase the load on the server is to increase the call frequency per user.

Per default, the property ***thinktime*** is set to 5 seconds. If a call lasts 100 ms, the user waits almost 98% of the time until the next call is sent to the server. If you decrease the value of ***thinktime*** to 1 second, almost five times as many calls are sent to the server in the same time.

An extreme configuration is to set ***thinktime*** to 0. Compared to the default setting, approximately 50 times as many calls are sent to the server calculating with a response time of 100 ms. Especially when using this setting or a low think time, communication errors can occur such as the following:

WSAEADDRINUSE: Address already in use

For an explanation of the problem, see SAP Note 734095. You can avoid this problem by extending the duration of a session. This is done by the property ***loopspersession***. You can span a session over 'n' loops by setting ***loopspersession = n***. Setting ***loopspersession = 0*** causes all steps to be executed in one session. This would also increase the call frequency of the test steps a little bit because no login and logoff calls are made in between.

If you set ***thinktime*** to a value of 1 second or greater, an optimization can be done on the server. For ***calltype=RFCStateful*** the parameter ***rfc/wait_rollout*** can be set to 0. The default value of this parameter is 500. This means that the system waits 500 ms to roll out the user context from the work process. However, because the think time is at least one second, waiting for 500 ms does not help at all. Therefore, it is better not to wait for the roll out if you want to achive a high load on the system.

9.3.2.3 Number of Users

The most natural way to increase the load is to use more users. However, it is also the most complicated one.

The number of simulated users is defined by the property ***clientnum.*** Up to 10.000 users can be specified.

However, if the number exceeds the value of some communication parameters, they have to be increased, too. The following parameter values might have to be adapted according to the value of ***clientnum***. To be on the safe side and to allow additional communication beyond the HA Test Tool, they should be bigger than the value of ***clientnum***.

Client Side	
-Djco.cpic_maxconv=n	For ***Calltype = RFCStateful:*** Maximum number of CPIC connections that JCo can handle. The default depends on the platform. Normally it has to be set if ***clientnum*** is bigger than 200. The phrase has to be included in the Java call string of the client: java ***-Djco.cpic_maxconv=n*** -cp …

Server Side	
gw/max_conn = n	Number of logical connections to a gateway, that is, the number of connected gateways and external programs.
gw/max_sys = n	Maximum number of Gateway clients.
rdisp/max_comm_entries = n	Number of communications from and to an instance (not including dialog users).
rdisp/tm_max_no = n	Number of connections to an instance, including both dialog users and interfaces.
The following server parameters might be of interest, too:	
rdisp/appc_ca_blk_no	Only for SAP NetWeaver releases < 7.40 Buffer for TCP/IP communication. Especially if the load gets very high in the system, more memory is used to queue the communication requests. In this case, the value can be increased approximately to the value of the parameters above. For more details, see SAP Note 3223.
rdisp/wa_ca_blk_no	Only for SAP NetWeaver releases >= 7.40 Communication buffer for TCP/IP and DIAG communication. In SAP NetWeaver 7.40, the buffer for TCP/IP communication (see ***rdisp/appc_ca_blk_no*** above) was merged with the buffer for DIAG communication. The buffer size should be

	sufficient for the HA Test Tool if no significant dialog usage is done in parallel. Otherwise the size has to be increased according to the recommendation given for ***rdisp/appc_ca_blk_no*** above.

In addition to the parameters above, there might be other limitations caused by the communication layer, especially TCP/IP. If you run into communication problems because of many users although the values of the parameters above are high enough, you should check whether there are limitations of the communication layer on your platform. An alternative would be to reduce the number of users so that the communication works again and increase the load by using the possibilities described in chapters 9.3.2.1 and 9.3.2.2.

In addition to the parameter setting, you have to be aware that the HA Test Tool creates four output files per simulated user on the client side plus several other files. You have to make sure that your operating system environment allows to use that many open file handles.

The memory footprint on the server side is increased by the amount of user contexts. You need to make sure that this does not cause memory shortage or swapping.